

Automating Data- and Code-Archiving Checks with Metacheck: A Replication and Extension of Cooper et al. (2026)

Daniël Lakens

Checking whether published articles archive usable data and code is time-consuming: even the lightest form of computational reproducibility checking, running an article’s own code on its own shared data, has been estimated to take around 24 minutes per article, and writing independent analysis code from scratch has been estimated at 2-25 person-hours per article. Cooper and the BES Data and Code Hackathon Group (2026) manually coded data- and code-archiving practices in 1861 papers published across the seven British Ecological Society (BES) journals between 2017 and 2024, using a two-day hackathon of 145 participants who followed a bespoke coding protocol. This manual effort is exactly the kind of task that automated tools such as Metacheck aim to support: extracting structured information about open science practices from the text and repositories linked to a scientific article, in a fraction of the time. We ran Metacheck’s `repo_check`, `data_check`, and `code_check` modules on the same corpus and compared its output against Cooper et al.’s human-coded ground truth for every construct where a direct, mechanical comparison is possible: data availability, archive location, data licensing, data and code download success, code archiving, and README presence. For each construct we report sensitivity, specificity, and two directional miss rates (how often a human coder caught something the tool missed, and vice versa), and, going beyond aggregate accuracy, we individually reviewed every disagreeing paper against Metacheck’s saved output, the paper’s own text, and, where relevant, a live call to the repository platform’s own API, to determine which side was correct and why. Agreement on whether data could be downloaded and used was 99.0%, and on whether code could be downloaded was 99.5%; agreement was lower and more variable for whether a repository could be found at all (83.9%), whether code was archived (87.4%), and whether a README was present (71.8%). Metacheck’s precision is consistently high across constructs — when it reports a repository, license, download, or README as present, that report is rarely spurious — but its recall is uneven and construct-dependent, driven by a mix of a repository-detection coverage gap (a real, working data or code citation on a platform or in a format

Metacheck does not yet recognise), a structural limitation in inspecting the contents of compressed archives, and at least one genuine definitional mismatch (platform-level description metadata a human coder reasonably reads as README-equivalent, which Metacheck’s stricter file-presence check does not). Our goal is not to replace manual coding of this kind, but to clarify, concretely and per construct, where automation can currently reduce the burden of large-scale meta-scientific data collection while maintaining comparable accuracy, and where it currently cannot, and why.

Introduction

The last two decades has seen a notable culture change around open science practices. Universities, journals, and funders increasingly expect researchers to engage in open science practices. One open science practice that has become especially widely required and adopted is data and code sharing. The goal of data and code sharing is to facilitate re-use, and to increase transparency. Funders have started to require data management plans that specify where and how data and code will be made available for re-use, and submission guidelines for journals often require that data and code underlying the claims in a manuscript are shared if possible. The British Ecological Society, for example, requires that “all data and code supporting the results in the paper must be archived in a public repository” (<https://www.britishecologicalsociety.org/publications/journals/author-guidelines/>).

Although the requirements to share data and code are clear, it is less clear who is responsible to check that authors actually follow through on these requirements. Manually checking data and code files is a time-consuming task that requires expertise about best practices in data and code sharing. Due to the diffusion of responsibility and the lack of resources to check whether scientific manuscripts follow best practices in open science, requirements to share data and code are often not followed, and if data and code are shared, best practices are often not followed. Cooper and the BES Data and Code Hackathon Group (2026b) assessed data- and code-archiving practices across the seven journals published by the British Ecological Society (BES). They found that most manuscripts in which data was used also shared the data, but code sharing was less common, and even when data and code were shared, best practices were often not followed. About a third of data and/or code repositories lacked a README file, code not always well-annotated, and code was not always reproducible (e.g., due to hard-coded file paths). These findings strongly suggest that it would be beneficial if data and code is reviewed before a manuscript is published.

Some journals manually check data and code before an article is published. This practice is most common in economics journals, where a dedicated Data Editor oversees the reproduction of results based on data and code (Vilhuber & Cavanagh (2025)). Manual checks are laborious and delay the publication process. It is therefore not surprising that researchers are exploring automated ways to check whether data and code are shared, and whether they are usable. For example, the Scalable Infrastructure for Validation of Computational Social Science Research

(SIVACOR) pilot project by the data editors of the American Economic Association examines whether it is possible to automatically verify the computational reproducibility of data and code. The extent to which automated processes can reduce the workload associated with checks of data and code sharing practices is an open question. Automation has the potential to reduce the workload of manual checks, but it is not clear whether and where automated tools can achieve the same level of accuracy as human coders.

In this paper we will examine how Metacheck, a novel open source tool that checks best practices in data and code sharing, might assist in the process of checking whether data and code are shared. We will compare the output of Metacheck against the manually coded ground truth of Cooper et al. (2026b), available at <https://github.com/nhcooper123/reproduce-reuse-recycle>, to determine where Metacheck can currently reduce the manual burden of large-scale meta-scientific data collection while maintaining comparable accuracy, and where it currently cannot, and why. It is clear the human coders exerted considerable effort over the 2-day hackathon to code the 1861 papers, where 145 people coded a median of 10 (min = 1, max = 68) manuscripts. Cooper et al.’s protocol covered the seven journals published by the British Ecological Society (BES), coding papers published between 2017 and 2024 on eight questions per paper. The team coded whether data and code were used and archived, where they were archived, whether the archived files could be located, downloaded, and opened, what file formats were used, whether a README was present and how useful it was, how complete the archived data were, how well-annotated the code was, and whether the data and code were citable (i.e., had a DOI and a licence).

This is precisely the kind of structured, repetitive, and time-consuming coding task that automated tools such as Metacheck (DeBruine et al., 2026) aim to support. If an automated tool could complete some or all of Cooper et al.’s eight coding questions at comparable accuracy to their trained human coders, it would substantially reduce the workload that is currently required to check if researchers follow best practices in data and code sharing. Metacheck (DeBruine et al., 2026) is a modular, open-source tool that uses text search, rule-based classification, API’s, and optional large language models to extract information about research practices from scientific manuscripts and their associated repositories. It contains three modules — `repo_check`, `data_check`, and `code_check` that perform automated checks that closely resemble the questions Cooper et al. (2026b) asked their human coders to answer.

In this paper, we use Cooper et al.’s openly archived data as ground truth (Cooper & the BES Data and Code Hackathon Group, 2026a), and for each of the eight questions in their protocol, we (1) summarize what Cooper et al. found, quoting their own text verbatim, and (2) examine, concretely and with reference to specific Metacheck module output fields, whether that question can currently be automated, partially automated, or not automated at all. Where automation is possible, we run the relevant Metacheck module(s) on the same 1861 papers and report the results. The results generated by Metacheck are completely automated, do not rely on AI, and can be completely reproduced by re-running the code, without any human input.

Unlike a validation study that reports a single accuracy percentage, our goal here is to make transparent, question by question, exactly where the current version of Metacheck can reduce

the manual burden of this kind of meta-scientific data collection, and where it cannot. As an automated tool, Metacheck follows FAIR principles, which state that “all research objects should be Findable, Accessible, Interoperable and Reusable (FAIR) both for machines and for people” (Wilkinson et al., 2016, p. 3). Human coders can retrieve information that is not accessible by a machine. For example, if authors state that the data is shared as supplementary material, the data is not FAIR, because a machine can’t find and access the datafile. Cooper and colleagues did not limit their coding to FAIR data, and as a consequence, Metacheck will always miss some information that human coders found. On the other hand, machines can find some information that human coders miss. For example, the Dryad data repository requires a CC0-1.0 license at deposit for every dataset with no author choice involved, and adds this information to the metadata. Metacheck retrieves this information, but the human coders were unable to locate license information by visually examining the Dryad repository. Throughout this project we will also reflect on practical challenges when developing software that attempts to automatically retrieve FAIR data, and the implications for best practices in data and code sharing.

Methods

We obtained the list of paper DOIs, journals, and years directly from the cleaned dataset in the public repository, and downloaded all articles through an R script. All articles were converted to structured text using GROBID 9.0, and converted to Metacheck paper objects. We ran the `repo_check` (locates and lists files in online repositories), `data_check` (classifies files and extracts information about tabular data), and `code_check` (checks R, Python, SAS, SPSS, Stata, Mplus, and MATLAB code files) modules to mirror to manual coding performed by Cooper and colleagues. Subsequently, we ran the `reproducibility_check` module to illustrate additional benefits of automation.

1. Does the paper use data/code? If so, are they archived?

Cooper and colleagues reported that “Of the 1861 papers in our dataset, 93% (n = 1735) used data and 90% (n = 1670) used code. Of these papers, 97% (n = 1690) archived their data, but only 35% (n = 577) archived their code”. The first two percentages are based on text in data availability statements and the method section. There are tools that automatically check data availability statements, such as OddPub (Riedel et al., 2020). Metacheck itself has an equivalent module, `open_practices`, which reads the full text and judges from the wording alone whether open data and/or open code was shared. These automated checks most closely follow the coding by Cooper and colleagues, in that data and code is classified as available irrespective of whether authors followed FAIR principles. Metacheck’s `repo_check` module takes a different approach and classifies data and code as available based on a linked repository that can be automatically accessed through an API.

We compared the automatic text-based judgements from OddPub and the Open Practices module to Cooper and colleagues’ human coding of whether data and code were openly shared (their `data_open/code_open` columns), treating “Yes” and “Yes, but not all files” as open and “No” as not open, and excluding papers where the coders used an ambiguous label (e.g., “Needs specific software or too large”) or left the item unscored. For data sharing (1552 papers with a clear human judgement), OddPub identified 80% of the papers the human coders judged to share open data (sensitivity), and correctly identified 45% of the papers judged not to (specificity), for an overall agreement of 80%. Metacheck’s `open_practices` module agreed with the human coders on data sharing 86% of the time, with higher sensitivity (86%) but lower specificity (9%) than OddPub. For code sharing (498 papers with a clear human judgement), the difference was larger: OddPub agreed with the human coders 44% of the time (sensitivity 44%, specificity 67%), whereas `open_practices` agreed 78% of the time (sensitivity 78%, specificity 0%). Both tools, in other words, are much more likely to say a paper shares open data or code than to say it does not; this corpus contains very few papers the human coders judged as clearly *not* sharing (11 for data, 3 for code), so the specificity estimates above are based on very small numbers and should be treated as indicative rather than precise. `open_practices` and OddPub both determine open data/code from the text of the article alone, and neither distinguishes between data and code repositories the way `repo_check` and `code_check` do below.

Cooper and colleagues found that 1690 papers archived their data and 577 their code, and in total, 1789 papers contain a link to a repository with either data or code. In total, Metacheck found 1539 papers that contained a link to a repository in the main text where it could verify that the repository contained files. Of these, 1436 papers contained data, 435 papers contained code, and 1488 papers contained either data or code according to `repo_check`.

Comparing these two “either data or code” judgements paper by paper rather than only as totals: the two sides agree for 1472 papers, and disagree for 333 (317 where Cooper’s coders found data or code that Metacheck did not, and 16 where Metacheck found data or code that Cooper’s coders did not). There are three reasons for the 317-paper gap.

First, for Metacheck data and code is findable and accessible if it is archived in one of the repositories that Metacheck supports (through its support for Dataverse, DSpace 6 and 7, Figshare, and DataOne, and dedicated support for OSF, Zenodo, ResearchBox, Dryad, ReShare, GitHub, GitLab, Mendeley Data, FSD, AsPredicted, and 4TU). Because many repositories used the same underlying API, Metacheck supports over 276 unique repositories (121 Dataverse installations, 64 DSpace 7 installations, 6 legacy DSpace 6 installations, 5 DataOne member nodes, and 69 Figshare-based hosts, plus 11 dedicated single-platform integrations). This is only a subset of all repositories used in manuscripts, and corpus-wide – out of all 1861 papers, regardless of what Cooper coded – `repo_check` found no repository at all for 322 papers. Of those, 267 are also a disagreement with Cooper’s coding (she coded that paper as having data or code; the remaining 55 papers are ones where Cooper agreed there was no data or code, so `repo_check` finding nothing there was not a disagreement and needed no further review).

Within the 317-paper gap identified above (Cooper found data or code that Metacheck did not), 267 papers trace to `repo_check` finding no repository at all. A further 50 papers in that same gap are the reverse situation: `repo_check` did find a real repository, but none of its files were classified as data or code by `data_check/code_check`. Together these two causes – no repository found at all, or a repository found but nothing inside it recognised as data or code – account for 317 of the 317-paper gap; the remaining 0 papers are explained by something other than a repository-detection failure (for instance, Cooper’s coders judging a paper to share data or code for a reason that does not rest on a repository citation at all, such as data embedded directly in the article’s own text).

We individually reviewed every one of these 267 papers by reading its own data-availability text and, where a specific platform was named, verifying the citation live against that platform (Appendix A.6’s `repo_not_detected_categorization.csv`; Table 1 below summarises the same review). This finds that 121 genuinely cite a repository platform Metacheck has no backend for at all, 19 cite a domain-specific bioinformatics accession (SRA/GenBank/BioProject, a distinct category from an ordinary data/code repository), 4 cite a platform Metacheck does support but failed to detect for a specific, confirmed reason (Appendix A.9 documents each one), and the remaining 123 split across several unrelated, individually-confirmed causes (no data-availability text was captured for the paper at all, a citation was truncated before any identifier appeared, data were explicitly withheld with no repository to find, or the paper’s own published citation pointed to an internal, non-public address) – so most of what this mechanical count calls “no repository found” is not “Metacheck cannot access this platform” at all. Many of these papers were coded by Cooper et al. as “Other repo/database” – their catch-all for anything outside their own named platform list – but this label is not itself evidence Metacheck lacks support for the repository: several “Other repo/database”-coded papers instead cite a real OSF, Dryad, Zenodo, or Figshare deposit that Cooper’s coders simply had no more specific box for, which Metacheck supports and finds correctly. An institutional or domain repository Metacheck does not yet support may well be genuinely FAIR (carrying a DOI, a persistent identifier, and machine-readable metadata) even so – being unsupported by one tool is not the same as being unFAIR, and Appendix A.6 discusses these 121 papers’ repositories in full, including which specific platforms recur often enough to be worth a dedicated backend.

Automated tools need to be able to sustainably support existing repositories. Registries of data repositories exist (e.g., <https://www.re3data.org>) but the registry does not contain information about the APIs used by each repository. Some repositories disappear, and new ones are added. Support is relatively straightforward if a repository uses an established API (e.g., Dataverse, Figshare, DSpace) but requires more effort if it uses a unique API (e.g., OSF). This raises the question whether the definition of ‘findable’ data should incorporate the effort required to support a repository in automated tools. It seems likely that data stored at more widely used repositories (e.g., Dryad, Dataverse) will be more findable by automated tools than a small custom data repository used by a handful of scientists, simply because of the costs associated with supporting all data repositories. Of course, data repositories can facilitate the findability of the data they host by contributing to open source software, and contribute code that supports their own platform.

Three further questions bear directly on how to interpret the gaps documented above. First, among papers Cooper and colleagues coded as archiving their data on a specific, real, named repository platform – Dryad, Figshare, Zenodo, or GitHub/GitLab, as opposed to her “Other repo/database” catch-all or the non-FAIR “Personal website”/“Supplementary materials” labels – how often does Metacheck actually find something? These 1404 papers are, by construction, genuinely FAIR-archived: a real, persistent identifier on a platform with a documented, machine-readable API. Metacheck found at least one file for 1349 of them (96.1%), and found nothing at all for the remaining 55.

Table 1: Table 1a. Detection rate among papers Cooper et al. coded as archived on a specific, named, FAIR repository platform.

Platform	n	Found	Missed	Pct found
Dryad	1,022	992	30	97.1%
Figshare	165	151	14	91.5%
Zenodo	210	200	10	95.2%
GitHub, GitLab, Codeberg or similar platform	57	56	1	98.2%

Detection is not uniform across these platforms (Table 1a). GitHub/GitLab and Dryad are found nearly every time; Figshare is found noticeably less often, consistent with the recurring Figshare-specific detection defects documented above and in Appendix A.9 (several distinct DOI-shape and URL-format gaps, not one single cause).

Second, for the 51 papers where `repo_check` found a real repository but neither `data_check` nor `code_check` classified any of its files as data or code, what were those files actually typed as? Across their 2614 files combined, 1825 (69.8%) are typed `unknown` – a file extension `data_check` does not recognise at all, rather than one it deliberately excludes – and a further 569 are typed `documentation`. Manually inspecting several of these papers’ own file listings finds this `documentation` classification is not always right: one paper’s repository (10_1002_2688_8319_12029) contains dozens of plainly numerical simulation-output files (e.g. `nodes_n.txt`, `t_10.txt`, `2K.txt`, `arcs0_nos_1n_v2.txt`) that `data_check` classifies as `documentation` purely because they carry a bare `.txt` extension, the same extension a genuine README or methods note would carry – `data_check` has no way, from extension alone, to distinguish a prose document from a plain-text numeric output file. This is a real, distinct limitation from the platform-detection gaps discussed above: the repository was found and its files were listed correctly, but a common, ambiguous file extension was classified in a way that undercounts the paper’s actual archived data.

Third, and related: how does `data_check` classify `.tif/.tiff` and ESRI Shapefile files specifically, since both are common formats for GIS/remote-sensing research data in ecology? Neither has a clean, unconditional rule the way `.csv/.fasta/.fastq` do. `data_classify_files()`

only recognises a `.tif` or `.shp` file as `data` via a secondary, folder-path-keyword fallback (e.g. a path containing a segment like `data/`, `spatial`, or `sdm`) – there is no rule based on the extension alone. A bare filename with no folder, or a folder named for something unrelated to data (e.g. “Priority maps”), falls through instead to `materials` (for `.tif`) or `unknown` (for `.shp`), even when the file is genuinely the paper’s data. Across the 22 papers where Cooper and colleagues’ own coding names `.tif` as (part of) the archived data format, 2159 of 2718 real `.tif` files found are correctly typed `data`, and 443 fall through to `materials`; across the 29 papers naming `.shp`, 66 of 150 real Shapefile-component files are correctly typed `data`, 84 fall through to `unknown`, and 11 of the 29 papers have every single one of their Shapefile components misclassified. This is a real, folder-naming-dependent detection gap distinct from every other cause discussed above – not a deliberate scope decision (unlike, say, excluding a `.jpg` photograph of a specimen, which genuinely is not a data table) – and, together with the missing-extension patterns found throughout this section, motivates the broader file-type-detection redesign proposed in Appendix A.10.

Comparing the manually coded results by Cooper and colleagues against Metacheck, 267 papers showed at least one “repository not detected” disagreement. The cause for every disagreement was categorized by reading the paper’s own extracted text and, where relevant, querying the cited platform’s own API directly. Table 1 summarizes the underlying reasons for a missed data repository by Metacheck.

Table 2: Table 1. Reasons why Metacheck Misses an Existing Repository.

Category	n papers
Existing repository not supported by Metacheck	121
PDF-to-text extraction cut the DOI off mid-string, or captured no data-availability text at all	36
PDF-to-text extraction inserted whitespace inside the DOI/URL	25
Domain-specific bioinformatics accession (SRA/GenBank/BioProject/MG-RAST)	19
Citation appears only in the reference list	11
Environmental Data Initiative, not supported	8
Data shared as supplementary files	8
NERC Environmental Information Data Centre, not supported	6
Confirmed to be genuinely inaccessible	6
PANGAEA data publisher for earth and environmental science, not supported	6
Not a data repository, CRAN package	5

Category	n papers
USGS ScienceBase, not supported	5
No repository cited; data explicitly withheld	4
Supported platform, but a confirmed Metacheck defect prevented detection	4
A private Figshare share-link with no resolvable article id	2
Paper's own published citation is unusable (e.g. an internal, non-public address)	1

Two categories are worth distinguishing explicitly from a genuine Metacheck defect, since the difference bears directly on whether this residual is fixable at all. `EMBARGOED_OR_RESTRICTED_AT_SOURCE` (confirmed by querying the repository's own API directly: Dryad's own dataset endpoint, for every one of these, returns an explicit "Identifier cannot be viewed. Either you lack permission to view it, or it is missing required elements" message, not a Metacheck-side error) and `TEXT_EXTRACTION_TRUNCATED` (the DOI is cut off mid-string in the paper's own extracted text, before it ever reaches any Metacheck detection logic at all) are not repository-coverage gaps; no platform-support fix could close either one, since the citation itself is unrecoverable or the data is genuinely withheld. `CRAN_CODE_PACKAGE` is a related but distinct case: the paper's own code is a real, working, publicly available CRAN package, correctly outside `repo_check`'s scope (CRAN is a code-package registry, not a data or code repository in the sense every other platform this package supports is), so this is a scope boundary rather than a defect. `SPLIT_DOI_EXTRACTION` (25 papers) is the same underlying mechanism documented for `data_availability`'s `other` bucket earlier in this manuscript, now confirmed to recur for Dryad DOIs specifically as well as the generic `doi.org` case already known, and is filed as a currently open, unfixed limitation in Metacheck's own text-extraction pipeline rather than in any individual platform backend. `EDI_ENVIRONMENTAL_DATA_INITIATIVE`, `EIDC_NERC`, `PANGAEA`, and `USGS_SCIENCEBASE` are each confirmed to be real, recurring, appropriately-scoped platforms with a genuine, documented public API (verified directly against each platform's own API documentation) — the largest, most tractable remaining coverage gaps this audit identified, each affecting enough papers on its own to be worth a dedicated backend rather than folding into the generic `UNSUPPORTED_REPO` bucket, which remains the largest single category (121 papers) precisely because it aggregates every platform that appeared only once or twice in this corpus, for which a dedicated backend would fix one paper at a time rather than a whole recurring class of citation. `SUPPORTED_PLATFORM_BUG` and `SUPPORTED_PLATFORM_MISSED` are a different kind of residual again: these papers cite a platform Metacheck already supports, so the fix is not new platform coverage but repairing a specific, already-diagnosed defect in existing support (Appendix A.9 documents each one individually).

Second, Metacheck does not recognize all files as data or code. Scientists use a great number of software solutions which create data files with different extensions and structures. Metacheck

currently recognizes only a subset of these as data or code files, which is a second cause for the difference between the human coded ground truth and the results by Metacheck. Third, Metacheck searches the full text for links to repositories. Some authors do not mention a data repository in the paper, but add a data citation. The link to the repository then appears only in the reference list, which Metacheck does not process. Although it is good practice to cite datasets to give credit to the researchers who created them, it improves the findability of data if links to data appear in the main text (E.g., in the data availability statement), in addition to a citation. This highlights how different goals (i.e., providing credit for data sharing, and finding data) can conflict. It might be possible to also scan all DOIs in a reference list, and retrieve which DOI's link to data, and subsequently determine whether that data is used in the manuscript, but this would require extensive software development and validation, while including a direct link to repositories is a well-validated and working solution.

As a further check on these causes, every paper where Cooper and colleagues' own coders recorded a repository link that Metacheck's data availability judgement disagreed with (213 papers, a differently-defined and only partly overlapping population from Table 1's 267 papers above, built independently and not cross-reconciled against it) was reviewed a second time from scratch, by a different method than Table 1: rather than the author reading each paper directly, this review was conducted by one independent AI agent (Claude, Anthropic) per paper, each instructed to locate the citation in the paper's full extracted text independently of both Cooper's recorded excerpt and Metacheck's own detector output, query the live repository page or API directly to check public accessibility, license, and file listing, and check Metacheck's own source code directly to determine whether a backend exists for the cited platform. (Three of the 213 papers were instead checked directly by the author, as a manual pilot before the agent-based pass; the remainder were not reviewed by a human afterward.) This method is stated explicitly because it differs from every other reviewed classification in this manuscript, and because an AI agent's live-fetch result and platform-support judgement carry a different, and currently less well-characterised, error profile than a human reviewer's. The full per-paper record – each agent's independently-found link(s), live access/license findings, and classification justification – is in `code/03_comparing_results/manual_repo_findability_audit.csv`, so any individual classification can be checked and disputed.

Of these 213 papers, 23 were confirmed to already be correctly detected by Metacheck with no real gap on review (the paper appeared in this list for an unrelated reason, such as a downstream disagreement after the repository was already found). A further 53 are gaps Metacheck could close without building any new platform support at all: either the platform is already supported but this specific institutional host or DOI prefix is not yet registered (29 papers, e.g. an unregistered institutional Figshare or Dataverse prefix), the platform is supported and the citation is clean but a specific, now-diagnosed defect prevents detection (6 papers), or the link is present in the extracted text but corrupted by a stray character inserted mid-string (18 papers). Together, this means Metacheck already finds, or could find with no new platform-support work, 76 of the 213 papers (36%).

The remaining 137 papers are not fixable this way. 119 genuinely cite a repository platform

Metacheck has no backend for at all (the largest single cause in this audit, consistent with Table 1’s UNSUPPORTED_REPO/NCBI_GENBANK/EIDC_NERC/etc. categories above, now independently confirmed paper by paper rather than inferred from Cooper’s own platform label). 5 have no trace of the repository link anywhere in Metacheck’s extracted text at all, confirmed by searching the full extracted string directly rather than any particular section. The remaining 13 are not a Metacheck limitation at all: live-checking the cited repository directly found it does not currently resolve to an accessible public resource – for instance, a paper’s own text stating a fixed embargo period – so no automated tool, however complete its platform coverage, could have retrieved this data at the time it was checked.

Table 3: Table 1b. Independent manual re-audit of every paper where Cooper et al.’s coders recorded a repository and Metacheck’s data availability judgement disagreed, classified by why Metacheck could or could not have found it.

Category	n papers
Metacheck has no backend at all for this repository platform	119
Metacheck supports this platform family, but this specific host/DOI-prefix/URL-shape is not yet registered	29
Already correctly detectable by Metacheck; no real gap found on review	23
The link is present in the extracted text but corrupted (e.g. a stray space split mid-DOI)	18
The cited repository does not actually resolve to a public, accessible resource when checked live (e.g. a stated embargo)	13
Metacheck supports this platform and the citation is extractable, but a specific, confirmed defect prevents detection	6
The link is absent from the extracted text altogether	5

The categories above describe whether Metacheck specifically can find each repository, not whether the repository itself is the kind of FAIR, machine-queryable resource that some automated tool, with the right platform-specific code, could retrieve. These are different questions: a platform **Metacheck** has no backend for at all could still expose a documented API that a different or future tool could use, and conversely a platform **Metacheck** does support could still be cited through a link that is not actually machine-resolvable (for example, a Figshare private share-link, which carries no persistent identifier and cannot be resolved through Figshare’s own documented API). To answer this second question directly, each of the same 213 papers was reviewed again, by the same AI-agent method as Table 1b but as a separate, later pass with a narrower and stricter question: does the cited repository (1) carry a real persistent identifier (a DOI, not merely a URL), (2) expose a documented,

machine-queryable API (a REST API, OAI-PMH, SWORD, or equivalent – not merely a web page a human can click through), and (3) resolve live to a publicly accessible record right now? All three were required for a **TRUE** verdict; this is independent of, and does not reuse the verdict of, the Metacheck-specific review above, though it reuses the same live-fetch evidence where that evidence already settled the question unambiguously.

Of the 213 papers, 151 (71%) were judged machine-findable in principle by this stricter definition: a real persistent identifier, a documented API, and live public access, all independently confirmed. This is a different, and intentionally stricter, number from the 76 figure above (what Metacheck can already find or fix without new platform work) – it is also a different, and broader, population than Table 1a/1b’s named-platform papers, since it includes platforms Metacheck does not support at all whenever those platforms turn out to have their own documented API. Indeed, of the 119 papers classified **PLATFORM_NOT_SUPPORTED** above – cases where Metacheck itself has no backend – 80 (67%) were nonetheless found to expose a real, documented API once checked directly (for example, USGS ScienceBase’s REST API and official `sciencebasepy/sbtools` client libraries, or an EPrints repository’s standard OAI-PMH and REST interfaces), meaning these are repository-coverage gaps specific to Metacheck’s current backend list rather than evidence the underlying data is unFAIR or unfindable by automated tools in general.

Of the 62 papers judged not machine-findable in this stricter sense, 36 have no documented API at all (only a human-browsable landing page, a metadata-export endpoint, or a private, unversioned share-link with no resolvable persistent identifier), and 60 could not be confirmed as currently, publicly accessible (either an explicit embargo or access restriction, or – for a number of US federal repositories including ScienceBase – a Cloudflare bot-mitigation challenge that blocked every automated HTTP client tried, a tooling obstacle rather than evidence the record itself is private). These two reasons are not mutually exclusive and so do not sum to 62.

2. Where were the data/code archived?

When exploring where data was archived, Cooper and colleagues coded the specific repository where data and/or code was stored. They observed that some data stored on GitHub was also stored on Zenodo, in line with best practices as GitHub is not a stable data or code repository.

Cooper et al. report Dryad as the dominant data repository (57%, $n = 1022$), with Figshare (9%), Zenodo (12%), and “various institutional and government repositories” (17%) making up most of the remainder (Cooper & the BES Data and Code Hackathon Group, 2026b, p. 1959). `mc_data_archive` and Cooper et al.’s `data_archive` were both codeable for 1370 papers; because this is a platform-name comparison rather than a boolean, we score agreement as at least one overlapping platform name between the two sides’ (possibly multi-valued) answers. They agreed on 1323 (96.6%), disagreed on 47 (3.4%).

Reviewing all 47 individually: 31 were `METACHECK_RIGHT` — almost entirely Cooper et al.’s coders using the catch-all “Other repo/database” label for a repository whose platform is, on the paper’s own text and URL, unambiguous (most often OSF, e.g. a Data Availability Statement reading “Data and code available from the Open Science Framework: <https://doi.org/10.17605/OSF.IO/PNJFW>,” coded “Other repo/database” rather than “OSF”). 7 were `COOPER_RIGHT`, the mirror image of a pattern that recurs across several columns in this comparison: a paper genuinely has two separate repositories, one holding code and one holding data, and `mc_data_archive` picked up the `code` repository’s platform because `repo_check` found that repository’s files but never located the data repository’s at all (a domain-specific database, an institutional repository, or a Dryad/Mendeley deposit named in the text but never fetched). 7 were `BOTH_DEFENSIBLE`: genuinely multi-repository papers where Cooper et al.’s single-choice coding and Metacheck’s semicolon-joined multi-platform answer are each a defensible summary of a paper that does not reduce to one archive — several of these distinguish a versioned Zenodo archive from a live, mutable GitHub development repository, with Cooper et al. and Metacheck picking different repos by design. The remaining 2 could not be resolved from the available evidence.

Limitation. The recurring code-repository-mislabelled-as-data-repository pattern reflects a genuine architectural gap rather than a text-parsing failure in any single instance: `repo_check` finds every repository a paper links, but the logic that decides which one to report as “the data archive” for `mc_data_archive` has no signal for “this repository actually holds the study’s data” beyond checking which repositories hold `data_type == "data"` files — and that check is itself blind to a repository whose only file is an unopened archive (see the `code_archived` discussion below for the same limitation’s larger effect there).

mc_data_availability. TRUE if `repo_check`’s `summary_table$files_n` is greater than zero for a paper — that is, if `repo_check` found at least one linked repository whose file listing it could retrieve, with at least one file in it, of any type (data, code, documentation, or unclassified). This rule is deliberately permissive: it answers “did we find a repository with something in it,” not “did we find a repository holding a dataset.” That distinction matters in practice (see Results below): a paper whose only linked repository is a pure-code Zenodo archive, with zero data-classified files, still scores `mc_data_availability = TRUE` under this rule, because the rule does not check `data_type`.

mc_data_archive. The archiving platform, derived by matching each linked repository’s `repo_url` against a fixed set of domain and DOI-prefix patterns: `osf.io` for OSF; `github.com/gitlab.com` for “GitHub, GitLab, Codeberg or similar platform”; `zenodo.org` or the `10.5281/zenodo` DOI prefix for Zenodo; `datadryad.org` or the `10.5061/dryad` DOI prefix for Dryad; `figshare.com` or the `10.6084/m9.figshare` DOI prefix for Figshare; anything else is labelled “Other repo/database.” When a paper cites more than one repository holding data-classified files, `mc_data_archive` lists every matching platform, semicolon-joined; Cooper et al.’s protocol instead asks for a single platform per paper, so a genuine multi-repository paper can show a “disagreement” that is really a completeness mismatch rather than a wrong answer (see Results).

mc_data_doi, mc_data_license. Read from `repo_check$repo_metadata`, a table populated only for the platforms whose API `repo_check` queries for metadata beyond the file listing itself (Zenodo, Dryad, Figshare, GitHub’s repository object, and — when `osf_license = TRUE` is set, as it was for every run reported here — OSF). `mc_data_license` is `TRUE` if at least one repository holding a data-classified file has a non-missing `license` value in this table; `mc_data_doi` is that repository’s own `doi` field. This scoping choice — checking only repositories that hold data — is deliberate and matches `mc_data_archive`’s own scoping, but it means a paper whose licensed repository holds no data-classified files (a common pattern when data and code sit in physically separate repositories; see Results) scores `mc_data_license = FALSE` even though a real, retrievable license genuinely exists for that paper’s overall archive.

`repo_metadata$license` is read verbatim from each platform’s own API response, with no rewriting or standardisation on Metacheck’s side — this means its format varies by platform (a bare SPDX identifier like "MIT" or "GPL-3.0", a human-readable string like "CC BY 4.0", or a full SPDX URL), and it can be identical across many repositories on the same platform when that platform’s own licensing policy is uniform, not because Metacheck is returning a cached or default value. Concretely: 1059 of the 1059 licensed Dryad repositories in this corpus (100.0%) carry the identical string, <https://spdx.org/licenses/CC0-1.0.html>, because Dryad requires CC0-1.0 at deposit for every dataset with no author choice involved; this value never appears for a non-Dryad repository in this corpus (0 occurrences among Zenodo, Figshare, GitHub, and OSF repositories combined), whose own license values vary appropriately by repository and platform.

mc_data_download. `TRUE` if at least one data-classified file has `data_check`’s own `tabular_usable == TRUE` **and** a non-missing `file_location` (the local path the file was actually fetched to). This is a two-part requirement, not download success alone: a file can download successfully and still fail this check if `data_check` judges its content unusable as a table (see the Excel multi-sheet limitation in Results). `mc_data_format` is the sorted, semicolon-joined set of file extensions among the files that passed this same `tabular_usable` check.

mc_any_readme. `TRUE` if `repo_check` classified at least one file, in any of the paper’s linked repositories, with `doc_role == "readme"` — a purely filename-based match (a base name starting with "readme", case-insensitive, or a package-level `ro-crate-metadata.json`), never a check of the file’s content. Critically, Metacheck records only **one** README signal per paper, computed across all of that paper’s repositories combined; it does not distinguish a README documenting data from one documenting code. Cooper et al.’s protocol, by contrast, asks two separate questions (`data_README`, `code_README`). We therefore compare against `cooper_any_readme`, defined as `TRUE` if Cooper’s coders answered “Yes” or “Quasi-README” to either of their two README questions for that paper — the only comparison that avoids double-counting the same single Metacheck signal against two different Cooper columns (an earlier, columnwise comparison inflated the disagreement count by 41% purely from this double-counting, before we corrected it; see Results).

mc_code_archived. TRUE if `code_check`'s file table contains at least one row for the paper at all — that is, if `code_check` classified at least one file (of any language) as code, regardless of whether that file was ever successfully downloaded. **mc_code_download** is the stricter follow-up: TRUE if at least one of those code-classified files additionally has a non-missing `file_location`. **mc_code_language** is the sorted, semicolon-joined set of `code_check$table$language` values among a paper's code files; language is assigned by `code_lang()`, which is content-aware rather than purely extension-based for polyglot formats (`.Rmd/.qmd` and `.ipynb` are opened and their code cells inspected; an ambiguous `.html` is content-sniffed).

mc_code_used. Always NA. No Metacheck module answers “did this paper use code” as a construct distinct from “is there a code file sitting in a linked repository” — the two questions collapse into one from Metacheck's perspective, since it has no way to determine from the manuscript text alone whether custom code was part of the reported methodology.

Columns Cooper et al. coded that this comparison does not attempt. Cooper et al.'s protocol also includes README/code usefulness ratings (a 1-10 human-judged scale), a four-point data completeness judgment, and a code annotation quality score. None of these has a Metacheck counterpart to compare against: `code_check`'s `percentage_comment` field measures comment density (the proportion of lines that are comments), a real and precisely computed metric, but not a judgment of whether those comments are actually informative, so we do not force it into a comparison against Cooper et al.'s quality scale here. These three constructs are discussed qualitatively, not quantitatively, in the Discussion.

Empirical Comparison

We compare Metacheck's `mc_`-prefixed columns against Cooper et al.'s own coded ground truth across the 1861 papers we were able to retrieve, for every construct where a genuine boolean or categorical comparison is possible: whether data are available at all, where data are archived, whether a data license is present, whether data download and code archiving, and README presence (collapsed across data and code, per the Coding section above). For each construct we report: (1) how many papers both Cooper et al. and Metacheck coded a value for (`n_both`, since neither side coded every paper — Cooper et al. did not code a “not applicable” paper on every sub-question, and Metacheck cannot code a paper whose PDF we could not retrieve or convert), (2) a 2x2 confusion table of agreement and disagreement, (3) sensitivity (of the papers Cooper et al. coded **Yes**, the percentage Metacheck also coded **Yes**), specificity (of the papers Cooper et al. coded **No**, the percentage Metacheck also coded **No**), and McNemar's test for the asymmetry of disagreement, (4) the two directional miss rates the reader is likely to care about most in practice: of the disagreements, what percentage are cases where Cooper et al. found something Metacheck missed, and what percentage are cases where Metacheck found something Cooper et al.'s human coders missed, and (5) a qualitative synthesis of every individually verified disagreement, drawn from a case-by-case review in which every one of the 794 reviewed disagreeing papers (of 794 total; see the note on the 0 not-yet-reviewed rows below) was checked directly against Metacheck's own saved module output, the paper's own extracted

full text, and — where a repository platform’s API was the evidence source — a live API call at the time of review, and assigned one of five verdicts: `COOPER_RIGHT` (Metacheck’s answer does not match the paper/repository), `METACHECK_RIGHT` (Cooper et al.’s coded answer does not match the paper/repository), `BOTH_DEFENSIBLE` (the paper genuinely supports both answers, most often because it cites more than one repository), `DIFFERENT_DEFINITION` (the two sides are answering different questions that happen to share a column label — see the README discussion below), or `UNCLEAR` (the available evidence does not resolve the disagreement either way).

Following a full corpus rerun against a further round of fixes (Appendix A.1), the disagreement set itself was rebuilt from scratch rather than assumed unchanged: `data/cooper_vs_recreated.RData` was regenerated from the fresh `repo_check/data_check/code_check` output, and a new disagreement worklist was derived from it using the same mechanical flagging rule as before. For every (paper, column) disagreement whose Cooper-coded value and Metacheck-derived value were both unchanged from the prior review round, the earlier verdict and its evidence were carried forward rather than re-derived. Every disagreement that was new, or whose Metacheck-derived value had changed as a direct result of the fixes, was reviewed from scratch, following the same evidence-source rule throughout: `data_availability`, `data_archive`, and `code_used` were checked against the paper’s own extracted text, since these are what Cooper et al.’s coders themselves read; `data_license`, `data_download`, `code_archived`, `code_download`, and `any_readme` were checked against Metacheck’s own saved module output (`repo_check`’s file listing and `doc_role` classification, `repo_metadata`’s license/DOI fields, `file_location`), since a paper’s own text essentially never states whether a README exists or a download succeeded. A live platform API call (Dryad’s dataset API, Zenodo’s records API, Figshare’s articles API, GitHub’s repository API) was used only where local evidence was insufficient to decide. Appendix A.1 documents the specific software defects this case-by-case process surfaced and fixed over the course of the project.

The numbers reported in this section reflect a SECOND such rebuild, following Appendix A.5’s seven further fixes — applied and reverified this time against the FULL 1,861-paper corpus (not just the previously-disagreeing subset A.5 itself checked), using the identical carry-forward procedure: `code_data_preparation/57_rebuild_worklist_carry_forward_post_a5.R` re-generated `cooper_vs_recreated.RData` from a fresh `repo_check/data_check/code_check` rerun of every paper, rebuilt the disagreement worklist, and carried forward each prior verdict wherever its underlying (paper, column, Cooper value, Metacheck value) triple was unchanged. 794 of 794 disagreements carried forward or were otherwise already resolved this way; the remaining 57 papers (87 individual construct-level disagreements, since a paper can be flagged on more than one column) were newly surfaced or changed by A.5’s fixes and were reviewed from scratch the same way as every earlier round, following the same evidence-source rule described above (`code_data_preparation/59_apply_a5_review_verdicts.R`, with the full per-row reasoning in `code_data_preparation/verdicts_a5_review.csv`). Every disagreement in the corpus — 794 of 794 — now carries a verdict.

This final round of review surfaced one further genuine software defect — and, unlike

every other defect this project fixed and moved on from, this one was caught, root-caused, fixed, and reverified within the same review cycle rather than left open. Appendix A.5’s R-package-source-tree exclusion (intended to drop a bundled dependency’s own source tree — `DESCRIPTION/NAMESPACE/R//man//tests/` — from a repository’s `data/code` listing, since that is reusable infrastructure rather than the study’s own analysis code) turned out to also strip a repository’s real analysis code whenever the paper’s own code was itself organised as an installable R package, which is common practice for ecological modelling and analysis tools shared alongside a methods paper. Confirmed across 16 real GitHub/CRAN-hosted repositories (`RevGadgets`, `BaBA`, `m2b`, `RRPP`, `actel`, `scampr`, `MINOTAUR`, `LadderFuelsR`, `biomonitorR`, `EcoEnsemble`, `fossilsim`, `frair`, `tRophicPosition`, `sabinaNSDM`, `bef`, and `phylomice`), `repo_check`’s file listing for each package’s repository contained only its `build/configuration/documentation` scaffolding (`.Rbuildignore`, `.gitignore`, `NEWS.md`, `README.md`, `cran-comments.md`, a bare `.Rproj` file) with the package’s actual `R/` source directory entirely absent — and, in several of these repositories, the package’s own bundled `data/` directory was stripped along with it, silently turning a genuine data deposit into an apparent absence of data rather than merely a wrong-but-visible code disagreement.

The fix distinguishes two structurally different cases that both fingerprint identically (a `DESCRIPTION+NAMESPACE` package at a repository’s root): a study that shares its own data and analysis code as an R package (`BaBA`’s tracked-animal datasets and behaviour-classification functions, for instance) versus a repository that happens to bundle a vendored copy of someone else’s general-purpose package alongside the study’s own files (confirmed against `MarshallLab/MGDrive`, a general-purpose mosquito gene-drive simulation package whose paper-specific fork nests the vendored package one level deep, with the paper’s own `Examples/` scripts and data sitting as siblings at the repository root). The corrected rule only excludes a root-level package’s files when nothing classifiable as real data or code survives outside it (checked with `data_classify_files()`, the same rules-based classifier `data_check` itself uses, so “real content” means the same thing everywhere in the pipeline); a package detected in a nested subfolder — the vendored-dependency shape — is still always excluded, since by construction there is always something else in the repository for a nested package to sit alongside. Reapplying this corrected rule to the 38 repositories flagged by a corpus-wide scan for the exclusion’s symptom (a repository whose only surviving files, after the old exclusion ran, were scaffolding) resolved 11 of the 16 confirmed cases; 5 remain flagged, but for an unrelated, pre-existing reason — a handful of these repositories’ cached downloads never retrieved the package’s `R/` source directory at all (a fetch gap, not an exclusion-logic problem), so the corrected rule correctly finds nothing outside the package to preserve because nothing outside the package was ever downloaded in the first place. That fetch gap is a separate, still-open limitation, tracked apart from the exclusion fix itself.

Is any data available at all?

Cooper et al. report that “97% (n = 1690) archived their data” among the 1735 papers coded as using data (Cooper & the BES Data and Code Hackathon Group, 2026b, p. 1958). `mc_data_availability` and Cooper et al.’s `data_availability` were both codeable for 1735 papers. They agreed on 1464 (84.4%), disagreed on 271 (15.6%). Sensitivity was 85.4% and specificity was 44.4%, McNemar’s $\chi^2(1) = 178.60$, $p < .001$. This is a considerably more balanced pattern than an earlier pass of this corpus found, because `mc_data_availability`’s own derivation was corrected during this project (see Coding section): it now reflects `repo_check` finding at least one linked repository with at least one file, rather than defaulting to a missing value whenever `repo_check` found none at all — so a paper with no detectable repository now correctly scores **FALSE** and enters this comparison, rather than being silently excluded from it.

Reviewing all 271 disagreements individually: 230 were **COOPER_RIGHT**, 8 **METACHECK_RIGHT**, 23 **DIFFERENT_DEFINITION**, and 10 **UNCLEAR**. The dominant cause among the **COOPER_RIGHT** cases (192 of 230) is `repo_check` never detecting a repository at all, even though the paper’s own text names one explicitly — most often a real, working DOI or URL on a platform `repo_check` has no dedicated backend for (institutional repositories such as DataSTORRE, DIGITAL.CSIC, or the Australian Antarctic Data Centre; domain-specific databases such as NCBI’s Sequence Read Archive/GenBank, the Knowledge Network for Biocomplexity, or Neotoma), or occasionally a citation malformed by PDF text extraction. A further 3 trace to a `repo_check` processing error logged internally (an R-level error raised while fetching an otherwise-correctly-detected repository, or the host platform being transiently unreachable during the corpus run) rather than a missed citation — a genuine, fixable software defect distinct from a coverage gap. Conversely, 8 disagreements were **METACHECK_RIGHT**: clear, working Dryad, Zenodo, Figshare, or GitHub data-availability statements that Cooper et al.’s coders appear to have missed or coded **No** in error. 23 were **DIFFERENT_DEFINITION** — most commonly a paper stating data are available “on request,” embedded directly in the article’s own Supporting Information, or withheld for confidentiality reasons, which Cooper et al.’s protocol appears to credit as “available” while `mc_data_availability` structurally cannot, since there is no externally checkable repository for `repo_check` to find. The remaining 10 could not be resolved from the available evidence.

Limitation. The mechanism responsible for `mc_data_availability`’s disagreements has changed since an earlier pass of this corpus: rather than a single, blanket rule failure that made a **FALSE** structurally impossible, the remaining gap is now dominated by `repo_check`’s coverage of repository platforms and citation formats — addressable by extending its URL-recognition patterns to more institutional and domain-specific repositories, rather than by changing the derivation rule itself.

Is the data licensed?

Cooper et al. report that 85% ($n = 1438$) of papers with archived data had a license (Cooper & the BES Data and Code Hackathon Group, 2026b, p. 1960). `mc_data_license` and Cooper et al.’s `data_license` were both codeable for 1292 papers. They agreed on 1237 (95.7%), disagreed on 55 (4.3%). Sensitivity was 99.2% and specificity was 26.2%, McNemar’s $\chi^2(1) = 21.02$, $p < .001$.

All 55 disagreements were individually resolved against the relevant repository platform’s own API, called live at the time of review. 45 were `METACHECK_RIGHT`: a real, retrievable license genuinely exists — most commonly CC0-1.0 for Dryad datasets (Dryad requires a license at deposit) or a depositor-selected “Other (Open)” tag for Zenodo records, or a GitHub repository license confirmed live via GitHub’s own repository API. 9 were `COOPER_RIGHT`, and the large majority share the same repository-scoping mechanism documented for `mc_data_archive` above: a licensed repository genuinely exists for the paper, but `data_check` classified zero of its files as `data`-typed (often because the only file is an archive never opened, or because the licensed repository is a separate code deposit rather than the data-holding one), so `mc_data_license`’s data-only scoping rule never reaches it; a smaller number trace to `repo_check` never detecting the licensed repository at all. The remaining 1 could not be resolved because `repo_check` found no `repo_metadata` row at all for the relevant repository.

Limitation. `mc_data_license` inherits `mc_data_archive`’s repository-scoping limitation, since both restrict themselves to “the repository holding data.” It remains one of the more reliably automatable constructs in this comparison in the `METACHECK_RIGHT` direction: every case where Metacheck found a license was independently reconfirmed against the same live API `repo_check` itself reads from.

Can the data be downloaded and used?

Cooper et al. report that, of archived datasets, “95% could be downloaded, and 91% could be opened” (Cooper & the BES Data and Code Hackathon Group, 2026b, p. 1959). `mc_data_download` and Cooper et al.’s `data_download` were both codeable for 1277 papers. They agreed on 1263 (98.9%), disagreed on 14 — a small enough count that every case was individually traced rather than summarized by category.

Of the 14 disagreements, 5 were `COOPER_RIGHT`: files that downloaded successfully but still failed `data_check`’s tabular-usability check for a real, distinct reason each time — a space-delimited scientific data format (climate-model output with a distinctive multi-part naming convention) the tabular reader does not recognise, an unusual two-row header layout, and other content-specific parsing gaps rather than a single repeatable bug. 4 were `DIFFERENT_DEFINITION`: the data genuinely downloaded and opens correctly in standard software, but `mc_data_download`’s own criterion (see Coding section) additionally requires `data_check`’s stricter `tabular_usable` flag, which does not always agree with “a human can open this file just fine.” 3 were

METACHECK_RIGHT: a downloaded file that, on direct inspection, was a serialized model-input dump with no valid row/column structure at all (so `tabular_usable = FALSE` was objectively correct, not a detection gap), a clean, working OSF link the paper’s own text supports with nothing to contradict it, and a KNB record a prior fix (scienceverse/metacheck#435) now correctly finds and downloads, which Cooper et al.’s No most likely reflects the pre-fix state of. The remaining 2 could not be independently confirmed, since the paper’s own text gives no detail on whether the data genuinely opens beyond a bare repository link — exactly the kind of live-repository fact a text-only check cannot resolve either way.

Limitation. This construct’s very small remaining disagreement count (14 papers, 1.1% of the codeable base) reflects fixes applied earlier in this project (a multi-sheet Excel reading limitation, scienceverse/metacheck#379; see Appendix A.1 for the full mechanism and before/after numbers from that specific fix). What remains is a heterogeneous residual of genuinely distinct format- and content-specific parsing gaps rather than one further systematic cause.

Is code archived, and can it be downloaded?

Cooper et al. report that “only 35% ($n = 577$) archived their code” among papers using code (Cooper & the BES Data and Code Hackathon Group, 2026b, p. 1958), and, among archived code, that most is on Zenodo or GitHub-family platforms (Cooper & the BES Data and Code Hackathon Group, 2026b, p. 1959). We report `mc_code_archived` and `mc_code_download` together, since the qualitative review of both surfaced overlapping causes.

`mc_code_archived` and Cooper et al.’s `code_archived` were both codeable for 1669 papers. They agreed on 1459 (87.4%), disagreed on 210 (12.6%) — the largest disagreement rate of any construct in this comparison. Sensitivity was 69.2% and specificity 97.1%, McNemar’s $\chi^2(1) = 100.12$, $p < .001$. `mc_code_download` and Cooper et al.’s `code_download` were both codeable for 392 papers (a much smaller base, since this sub-question only applies when code was coded as archived in the first place); they now agree on 390 of 392 (99.5%), with 2 disagreements remaining.

`code_download`’s current near-total agreement reflects a substantial, multi-stage improvement over the course of this project, documented fully in Appendix A.1 (from 66.2% agreement before any fix, through successive intermediate rounds, to the current 99.5%). `code_archived` moved much less across the same fixes, consistent with its dominant causes being ones those fixes did not target.

All 210 `code_archived` disagreements and the 2 remaining `code_download` disagreements were individually resolved. `code_archived`’s 158 COOPER_RIGHT cases are genuinely multi-causal:

1. **Code bundled inside the same repository as the data, with code_check finding no file to classify as code there (23 cases).** A recurring, precisely worded pattern: a Data Availability Statement reading, for instance, “Data and R code available from the Dryad Digital Repository” — the paper’s own text explicitly confirms code is bundled with

the data in one deposit, `repo_check` found that repository, but `code_check` classified none of its files as code (most often because the only file is an archive never opened, the same zip-peek limitation described next).

2. **`repo_check` never opened a zip/tar archive holding the paper’s code (0 cases).** Metacheck does not inspect an archive’s contents by default; a Zenodo record whose only file is a GitHub-integration snapshot (filename pattern `owner/repo-vX.Y.Z.zip`) is a recurring, easily recognisable instance of this.
3. **`repo_check` never detected the repository at all (43 cases),** despite the paper’s own text explicitly naming it — the same platform-coverage and citation-extraction gap documented for `data_availability` above.
4. **A `repo_check`-side processing error, logged internally (1 cases)** — the repository was correctly identified but its fetch failed with an R-level error or a transient host outage during the corpus run.
5. **A more heterogeneous remainder (86 cases),** including code provided only as journal Supporting Information rather than in any linked repository, citations to a third-party dependency’s repository rather than the authors’ own code, and `repo_check`-side file-classification errors (e.g., shapefile metadata files misclassified as code).
6. **The R-package-source-tree exclusion defect discussed above (5 cases remaining),** for the unrelated, pre-existing fetch-gap reason described there rather than the exclusion logic itself.

A smaller number of `code_archived` disagreements ran the opposite direction (29 `METACHECK_RIGHT`): 10 are real, substantial, multi-file GitHub or Zenodo code repositories that `code_check` correctly found and classified, coded No by Cooper et al. — a large, unambiguous set of named analysis scripts is not the kind of borderline case a coding protocol’s edge cases would plausibly explain, so these look like straightforward human-coder misses; a further 9 are papers whose own text and Data Availability Statement mention no code, script, or software at all, so Cooper et al.’s Yes has no traceable basis in the paper’s own evidence. 5 were `DIFFERENT_DEFINITION`: code described only as pseudocode or a script embedded in the journal’s own Supporting Information, which Cooper et al.’s coding appears to credit as “archived” while `code_check` structurally cannot see it (it is not in any linked repository). The remaining 18 were `UNCLEAR`, generally because the extracted paper text contained no relevant statement either way.

Of the 2 remaining `code_download` disagreements, one is `METACHECK_RIGHT`: real, verified downloaded code files (an OSF repository’s 9 code files, each independently confirmed with a non-missing `file_location`) that Cooper et al.’s coding missed. The other is `COOPER_RIGHT`: all 3 code files `code_check` found for this paper’s repository have a missing `file_location` — the same archive-member download-failure pattern traced and largely fixed elsewhere in this project (scienceverse/metacheck#429), affecting a paper that fix’s own rerun did not happen to cover.

Limitation. `code_archived` remains the construct in this comparison with both the highest disagreement rate and the most heterogeneous set of causes. The single largest cause (code

co-located with data in a repository `code_check` cannot classify, whether because the file is an unopened archive or for another reason) points to the same underlying zip-peek limitation affecting `data_archive` and `data_license` above; the second-largest (repository detection coverage) is the same gap documented for `data_availability`. `code_download`'s gap, by contrast, has narrowed to 2 remaining cases after the fixes documented in Appendix A.1 — this construct is, at present, essentially resolved.

Is there a README?

Cooper et al. report a README present for “66% (n = 1114) and 61% (n = 351) of the papers with archived data or code, respectively” (Cooper & the BES Data and Code Hackathon Group, 2026b, p. 1959). As described in the Coding section, Metacheck records a single README signal per paper (computed once, across every linked repository combined), so we compare it against `cooper_any_readme` — TRUE if Cooper et al. coded “Yes” or “Quasi-README” on either their `data_README` or `code_README` question. `mc_any_readme` and `cooper_any_readme` were both codeable for 1726 papers. They agreed on 1242 (72.0%), disagreed on 484 (28.0%) — the largest raw disagreement count of any construct in this comparison, though, as with `code_archived`, its composition changes the interpretation substantially. Sensitivity was 62.8%, specificity 95.5%.

Of the 484 disagreements, 23 were `METACHECK_RIGHT` on review — genuine cases where a real README file exists in a repository `repo_check` listed, and Cooper et al.'s coders missed it. 155 were `COOPER_RIGHT`, dominated (116 cases) by the same repository-detection gap documented for `data_availability` and `code_archived` above — `repo_check` never located a repository the paper's own text explicitly names, so no README evidence could ever be recorded — with a further 0 cases where the repository was found but its only file is an unopened archive, and 2 cases tracing to a `repo_check` processing error. The remaining 12 were `UNCLEAR`.

The single largest verdict category, however, is 294 `DIFFERENT_DEFINITION` cases (60.7% of all disagreements, well ahead of the `COOPER_RIGHT` count above) — not a detection failure on either side, but the two coding approaches answering genuinely different questions that happen to share Cooper et al.'s column label:

- **Dryad's abstract field (the largest single cause, 172 cases).** Dryad's own dataset API exposes exactly one free-text field, `abstract`. Checked directly on real Dryad datasets from this disagreement set, this field is consistently the paper's own scientific abstract — never file-by-file documentation of what the archived files contain. Cooper et al.'s coders, reading the same field on the live Dryad page, appear to have coded this abstract as satisfying “README present,” a defensible reading of their own protocol given what Dryad actually surfaces to a human visitor; `repo_check`'s `doc_role` classification, which requires an actual downloadable file named like a README, never counts it.
- **Zenodo's and Figshare's description fields (88 cases).** Unlike Dryad's abstract, these platforms' `description` fields are genuinely mixed in content — checked directly

on records from this disagreement set, real file-by-file documentation appears in some cases, a study summary in others, contact metadata in others, and an empty field in others. No single, reliable rule (treat the description as a README, or don't) would correctly resolve every case.

- **An archive-only repository whose single compressed file `repo_check`'s remote listing could not resolve (16 cases).** Any README bundled inside such an archive is invisible to the README check even when `repo_check`'s own zip-listing feature is enabled, for the same family of archive-access limitations discussed above.
- **A smaller remainder (18 cases)** not matching any of the three patterns above.

Limitation, and a genuinely different framing than the other constructs in this comparison. Unlike every other column reported here, `any_readme`'s disagreement is overwhelmingly not a case of either side being wrong. It is a case where Cooper et al.'s human coders, reading a live repository page, correctly used whatever free-text field that platform's own interface surfaced to them as evidence of a README-like description, while Metacheck's `doc_role` classification asks a narrower, more conservative question (is there a downloadable file that is actually named like a README) that structurally cannot see a platform's own abstract or description metadata at all. Resolving this gap in Metacheck's favor is not simply a matter of relaxing a filename rule: Dryad's abstract field is confirmed never to be README-equivalent content (so counting it would introduce false positives), while Zenodo's and Figshare's description fields are confirmed to be genuinely mixed (so counting them uniformly would introduce a different, more Zenodo/Figshare-specific rate of false positives). A reliable fix would need to distinguish, likely with a content classifier rather than a presence check, which platform-level description fields actually contain file-documentation content, on a case-by-case rather than platform-wide basis. ## Is a comment count a proxy for comment quality?

Cooper et al.'s protocol includes a construct no Metacheck module attempts to answer directly: `code_annotation_scale`, a human coder's 1-10 judgment of how well a paper's code is commented, coded for 543 of the 1861 papers. Metacheck's `code_check` module does not judge comment quality, but it does mechanically count comments: for every code file it can download and parse, it records `comment_lines`, `code_lines`, and `percentage_comment` (comment lines as a share of all lines). This raises a genuinely exploratory question distinct from every comparison above, none of which asked whether an automatable signal could stand in for a judged one: if a simple, free line-count already tracked Cooper et al.'s human quality rating closely, that would suggest the more expensive human judgment adds comparatively little beyond what counting could already tell us; if it does not, that would suggest the human coders are picking up on something a mechanical count cannot see.

We aggregated Metacheck's per-file comment counts to one value per paper — total comment lines divided by total lines across every code file `code_check` could download and parse for that paper — and compared it against Cooper et al.'s `code_annotation_scale` for the 376 papers where both were available (of 543 papers Cooper et al. rated and 432 papers Metacheck could compute a comment percentage for at all; the smaller overlap reflects that Metacheck's

count requires at least one code file to have actually downloaded and parsed, which, as the sections above document, is itself not guaranteed).

The correlation is weak but not zero: Pearson’s $r = 0.20$, 95% CI [0.10, 0.29], $p < .001$; Spearman’s $\rho = 0.17$, $p = 0.001$. Using the raw comment line count instead of the percentage gives essentially the same result ($\rho = 0.14$, $p = 0.008$), so this is not an artifact of how the percentage was computed. As a check on whether longer codebases are simply commented differently regardless of quality, we also correlated total code lines against percentage of comments directly: longer codebases have a modestly lower comment percentage ($\rho = -0.31$, $p < .001$), consistent with comments not simply scaling with code length, and not something that would by itself explain the annotation-scale correlation being as weak as it is.

Interpretation. A correlation of this size — roughly 4% of variance explained on the Pearson estimate — means a paper’s comment density is a poor stand-in for a human coder’s quality judgment: knowing what fraction of a codebase’s lines are comments tells us very little about whether a coder would call those comments good annotation. This is exploratory, not a claim about causation or about which measure is “right” — the two are answering different questions by design, and a low correlation between them is not evidence that either is wrong. But it does directly speak to the question motivating this comparison: for this specific construct, an automatable proxy does not substitute well for the human judgment call, at least not the simplest version of that proxy (line-count percentage). A single well-placed sentence explaining a complex analysis counts as excellent annotation on Cooper et al.’s scale and as a single line on Metacheck’s; ten lines of boilerplate restating variable names count as heavily annotated by line count and would plausibly score poorly on a human quality read. Comment content — whether a comment explains a non-obvious decision, restates the obvious, or is simply absent — is the kind of thing an LLM-based classifier could more plausibly approximate than a line count can, but that would need its own validation against this same ground truth before being trusted, following the same case-by-case standard used throughout this manuscript rather than assumed from the fact that an LLM could be asked to try.

Results you get for free alongside Metacheck

Every comparison above answers a question Cooper et al.’s protocol already asked. Running `repo_check`, `data_check`, and `code_check` on a corpus also produces a substantial amount of additional, structured diagnostic information that no item in Cooper et al.’s protocol asks for at all — not because Metacheck was built to exceed their protocol, but because the same file listing, the same downloaded code, and the same parsed spreadsheets that answer “is there a repository” or “is there a README” also support other checks the modules perform as a matter of course. This section reports what that additional output looks like at corpus scale, computed entirely from the same saved module output as the comparison above (`res_repo_check.RData`, `res_data_check.RData`, `res_code_check.RData`) — no additional module run was needed.

Code portability and reproducibility signals

`code_check` parsed 9202 code files across 435 code-bearing papers. Three checks bear directly on whether that code would actually run for someone other than its original author. (A systematic validation of these checks against real corpus files, reported in Appendix A.3, identified and fixed five false-positive/false-negative mechanisms; the numbers below reflect the corpus as run after those fixes.)

- **Missing referenced files.** For every code file, `code_check` extracts every filename the code itself tries to open (`read.csv(...)`, `source(...)`, and equivalent calls in each supported language) and checks whether that name appears anywhere in the same paper’s repository listing. 1121 files (12.2% of all analysed files), across 221 papers (50.8% of code-bearing papers), reference at least one file that could not be found. This does not necessarily mean the code is broken — a file might exist under a name the regex-based extraction did not match exactly, or might be generated by an earlier step in the same pipeline — but it is a direct, corpus-wide signal of how often “the code plus the shared data” does not obviously line up as a complete, runnable unit.
- **Hardcoded absolute paths.** 430 files (4.7%), across 112 papers (25.7%), contain an absolute file path (e.g. `C:/Users/...` or `/home/...`) rather than a path relative to the project. Code written this way will not run on any machine but the one it was written on without manual editing.
- **`setwd()` calls.** 409 files (4.4%), across 100 papers (23.0%), call `setwd()` to hardcode a working directory. Combined with hardcoded absolute paths (which frequently co-occur with a `setwd()` call to the same machine-specific location), the two portability problems together affect 269 of the 435 code-bearing papers (61.8%) — meaning more than half of the papers whose code we could inspect at all have at least one file that will not run as shared, on someone else’s machine, without modification.

Comment density (mechanical count)

As reported above in the comment-quality correlation, `code_check` records `comment_lines/code_lines` per file. Corpus-wide, comments make up 19.0% of all code lines. 1063 files (13.8%) have zero comments at all — of which 238 of 337 zero-comment Python files (70.6%) are in fact documented via a docstring rather than a `#` comment, a distinction `code_check` now also reports separately (`has_docstring`, Appendix A.3), so are not genuinely undocumented despite scoring zero on this mechanical count.

Import/library organization

Where a file loads more than one package, `code_check` also records how far apart those loading lines are (its own documented threshold is more than a few lines apart counting as “scattered” rather than grouped at the top of the file). 1013 of 4172 such files (24.3%) load their packages

in a scattered rather than grouped fashion — a minor readability signal, not a correctness one, but one a reader piecing together a script’s dependencies would find genuinely useful.

Parse errors

Of the 4938 R-type code files (`.R`, `.Rmd`, `.qmd`) that were both locally available and actually reached a call to `parse()`, 227 failed to parse as valid code in their declared language at all (4.6%) — a strong, unambiguous signal (not a stylistic judgment) that something is wrong with either the file itself or Metacheck’s identification of its language. (Only R-type files are eligible for this check at all; a file that was never downloaded, or that errored before reaching the parse step itself, is excluded from this denominator rather than counted as a pass — see Appendix A.2 for the full derivation and the two distinct causes behind the files excluded this way.)

Package version pinning

`code_check` also checks whether a repository pins the exact R/package versions its analysis depended on — an `renv.lock` file, a `sessionInfo()`/`session_info()` text dump, or a `groundhog/checkpoint` date-pinning call in the code itself (a bare `library(groundhog)` does not count; the pinning call itself must be present). Of the 435 code-bearing papers, 22 (5.1%) have version pinning present, recorded per paper as `code_version_pinned` in `code_check`’s own summary output (see Appendix A.2 for how this per-paper figure was recovered).

Spreadsheet and column-level data-quality findings

`data_check` inspects every tabular data file it can download and parse, both at the file/sheet level (is this a rectangular table at all, is the header on the first row, does the file use colour or merged cells to encode information Metacheck cannot read from a plain CSV) and at the level of individual columns (does a column look like it holds personal information, geographic coordinates, inconsistent category casing, numbers stored as text, or values in a legacy character encoding). Across the corpus, these checks produced 34799 findings, spanning 2933 distinct files. The most frequent, in descending order:

Finding	Count
Empty column	19,526
Numeric as text	4,611
Problematic column name	3,105
Colliding column names	1,591
Empty or unnamed columns	1,362

Finding	Count
Personal info (column name)	1,110
Whitespace	661
Geographic coordinates	603
Case issues	575
Empty rows	467
Colour coding	431
Merged cells	207
Free-text (may hold PII)	150
Mixed encoding	132
Un-inspectable (.xls)	72
Header not on first row	66
Personal info (values)	60
Not a rectangular dataset	43
Unreadable	27

Two of these are worth calling out specifically because they bear on participant confidentiality rather than only on tidiness: the two personal-information checks (by column name and by column values) and the “free text that may hold PII” check together flag 1320 findings across the corpus — columns or values that look, on their face, like they could contain names, contact details, or other identifying information in a file the authors have made publicly downloadable. Metacheck does not verify whether this is a genuine confidentiality problem (a column named “participant_id” that holds only anonymous sequential integers would also match on name alone), but a count this size suggests a possible participant-privacy check would find comparable value to the correctness-oriented deposit review Cooper et al.’s protocol already performs.

File-naming conventions

`repo_check` checks every listed file’s name against a set of FAIR-data-adjacent naming conventions: no spaces or special characters, an actually-classifiable name (not left as `data_type == "unknown"`), consistent casing, zero-padded sibling numbering, valid dates, and several path-length budgets. Across the corpus, this produced 64960 naming issues, 36856 of them the more serious “bad” severity (something that actually breaks: spaces, special characters, diacritics, an unclassifiable file, an invalid date, or an over-long path) and 28104 the softer

“suggestion” severity (a real convention worth following, but nothing in Metacheck’s own classification depends on it). 733 distinct papers (47.6% of the 1539 papers with a discoverable repository) have at least one naming issue of some kind. The single largest contributor by far is “unclassifiable” file names (23539 occurrences) — files whose name gives Metacheck’s classifier no signal at all about what kind of content they hold, which is itself a legibility problem for a human reader trying to navigate the repository, independent of anything Metacheck specifically failed to detect.

Why this section exists. None of the findings above were validated against Cooper et al.’s human coding, because Cooper et al.’s protocol did not ask about any of them — there is no ground truth to check against, and no sensitivity/specificity/agreement-rate analysis is possible for a construct nobody coded. We report them here, separately from the validated comparison above, specifically so that this distinction stays clear: these are Metacheck’s own outputs, useful in their own right for a researcher curious about a specific repository’s quality, but not independently verified the way every number earlier in this manuscript was. A natural next step, beyond the scope of this paper, would be a smaller, targeted validation of a sample of these specific findings (are the “unclassifiable” file names really unclassifiable to a human reader too; do the flagged personal-information columns really hold identifying information) using the same case-by-case standard applied throughout the rest of this manuscript.

Discussion

Table 0 summarizes the empirical comparison reported above: agreement rate, sensitivity, specificity, and the two directional miss rates for every construct where Metacheck and Cooper et al.’s coding can be directly compared.

Table 5: Table 0. Empirical agreement between Metacheck and Cooper et al.’s human-coded ground truth, for every comparison column. “Cooper caught, MC missed” and “MC caught, Cooper missed” are percentages of the disagreements only (they sum to 100% within a row), not of the full sample.

Construct	n	Agree %	Sensitivity %	Specificity %	Cooper caught, MC missed %	MC caught, Cooper missed %
Data availability	1,735	84.4	85.4	44.4	90.8	9.2
Data licence present	1,292	95.7	99.2	26.2	18.2	81.8
Data downloadable and usable	1,277	98.9	99.1	0.0	85.7	14.3
Code archived	1,669	87.4	69.2	97.1	84.8	15.2
Code downloadable	392	99.5	99.7	0.0	50.0	50.0

Construct	n	Agree %	Sensitiv- ity %	Speci- ficity %	Cooper caught, MC missed %	MC caught, Cooper missed %
Any README present (data or code, collapsed)	1,726	72.0	62.8	95.5	95.5	4.5
Archive platform (overlap match, not simple boolean)	1,370	96.6				

Two patterns are visible across every construct in Table 0. First, the size and direction of the sensitivity/specificity gap now varies considerably from construct to construct, and reviewing every individual disagreement, rather than stopping at the aggregate numbers, shows why: the asymmetry has a different cause in each column, and the cause matters more than the raw percentage for deciding what, if anything, to fix. `data_availability`'s specificity (44.4%) is far from the zero an earlier pass of this corpus found, precisely because the derivation-rule fix described in the Coding section gave it a genuine ability to disagree with Cooper et al. in the FALSE direction at all; what remains is a repository-coverage gap (see above), not a structural inability to ever produce a negative. `code_download`'s specificity is still 0.0%, but on a base of only 2 remaining disagreements this reflects a small handful of unresolved cases, not a systematic pattern; `data_download` currently has no codeable Cooper-No case at all, so its specificity is undefined rather than low. `code_archived` shows the most pronounced asymmetry of any boolean construct (sensitivity 69.2%, specificity 97.1%) and is genuinely multi-causal, spanning a real content-location mismatch (code bundled with data in a repository `code_check` cannot classify), a zip-peek limitation, a repository-detection gap, and a real “not in any repository at all” category (journal Supporting Information) no repository-scanning approach can close. `any_readme`'s asymmetry is not primarily attributable to either side being wrong at all: 60.7% of its disagreements reflect Cooper et al.'s human coders correctly reading platform-level metadata (a Dryad abstract, a Zenodo or Figshare description) that Metacheck's stricter, filename-based README check was never designed to treat as equivalent evidence, and where doing so uniformly would, on the evidence collected here, introduce as many new false positives (Zenodo/Figshare) as it resolved.

Second, precision in the direction Metacheck does commit to an answer remains generally high, though no longer universal: positive predictive value was above 97% for five of the six boolean constructs (`data_availability` 98.3%, `data_license` 96.4%, `data_download` 99.8%, `code_download` 99.7%, `any_readme` 97.3%), meaning that when Metacheck reports a repository, license, download, or README as present, that report is rarely spurious. `code_archived` is the exception: its PPV, 92.6%, is meaningfully lower, because 29 of its 210 disagreements are cases where `code_check` correctly found real code files that Cooper et al.'s coders missed or judged not to count — a genuine human-coder miss, not a Metacheck false positive, but one that lowers this specific metric regardless of which side is right. The asymmetry in this comparison therefore remains overwhelmingly a recall problem for most constructs —

Metacheck under-detects relative to a human coder who can read a repository’s page, follow an ambiguous citation, or interpret an unusual sentence structure — but `code_archived` shows this is not universal: a `TRUE` from Metacheck is a highly trustworthy signal for five of the six boolean constructs, while for `code_archived` specifically, a nontrivial share of Metacheck’s `FALSE` answers reflect a real detection gap rather than code genuinely being unarchived.

Three constructs in Cooper et al.’s protocol have no Metacheck counterpart at all and were therefore not part of the quantitative comparison above: `README/code usefulness` (a human-judged 1-10 scale), `data completeness` relative to a paper’s reported analyses, and `code annotation quality` as distinct from comment density. These are precisely the questions where Cooper et al.’s human coders had to read and interpret content, not merely detect its presence, and they mirror a pattern visible even within the constructs we could compare: Metacheck’s underlying modules answer structural questions (does a file with this name exist, did a download succeed, does a table parse) reliably, and answer content-interpretation questions (is this description a real README, is this the paper’s actual dataset rather than simulation output, is this repository the one holding the data rather than the code) far less reliably, precisely because those questions require reading and judging content rather than classifying structure. Large language models are a plausible route to closing part of this gap for the constructs Metacheck cannot currently address at all, and could plausibly also help with several of the misclassification patterns documented above (distinguishing a Dryad abstract from file documentation, or a simulation-output file from raw collected data) — but any such extension would need to be built, validated against Cooper et al.’s own ground truth using the same case-by-case standard applied here, and reported with a known error rate, rather than assumed to work from the fact that an LLM could plausibly be prompted to attempt it.

A recurring, cross-cutting finding worth naming explicitly is the data-versus-code repository conflation pattern: a genuinely common paper structure — one repository holding the study’s data, a separate repository (often GitHub) holding its analysis code, sometimes with the data-holding repository never located by `repo_check` at all — appears as the dominant or a substantial cause of disagreement in `data_archive`, `data_license`, and `code_archived` alike. This is not three independent bugs; it is one architectural gap (Metacheck’s repository-scoping logic assumes each construct maps cleanly onto “the” repository for a paper) surfacing across every construct that depends on correctly identifying which repository is relevant. It remains, after every fix applied over the course of this project, the single largest reason Metacheck and Cooper et al. disagree: repository non-detection alone accounts for 192 of `data_availability`’s 230 `COOPER_RIGHT` disagreements, 43 of `code_archived`’s 158, and 116 of `any_readme`’s 155 — a paper’s own text names a real, working DOI on a mainstream platform (overwhelmingly Dryad, Zenodo, or Figshare), and `repo_check` never queries it at all. Addressing the conflation and non-detection gaps once, at the level of how Metacheck decides which repository “counts” for a given question and which platforms it recognises, would likely resolve a meaningful fraction of the remaining disagreement across several constructs simultaneously, rather than requiring separate, construct-specific fixes.

Twelve individual software defects identified over the course of this project were reported

upstream, fixed, and reverified against a full corpus rerun for this manuscript. Five were found during the original review: the Excel multi-sheet `tabular_usable` limitation underlying most of the original `data_download` disagreement (scienceverse/metacheck#379, resolved: agreement rose from 93.5% to 99.1%), the Dryad unauthenticated-HEAD-request gap affecting both data and code downloads (scienceverse/metacheck#380), the `.zip-peek` authentication gap for archive-only repositories (scienceverse/metacheck#378), the `code_check` zip-discovery architectural gap surfaced while investigating the Zenodo data-download fix (scienceverse/metacheck#383), and a silent-truncation bug in the byte-fetch path that #383's fix newly exposed (scienceverse/metacheck#384): the function decompressing a zip member's bytes called an underlying library routine with no size hint, silently capping any member over 32,768 bytes at exactly that length with no error, rather than the size-aware call the library's own exact-size path rejects for an unrelated reason. Fixing #384 closed the great majority of the gap #383 had left open: `code_download` agreement rose from 66.2% (before #379/#380/#383) to 89.7% (after them) to 99.5% (after #384) — a two-stage improvement of over 30 percentage points overall, the largest of any construct in this comparison, and a level of agreement that has held with only 2 disagreement remaining across every subsequent fix and rerun described below.

A further seven fixes (Appendix A.5) — Zenodo reliability, wider repository-platform coverage, and a rate-limit handling defect — were applied and reverified against the entire 1,861-paper corpus rather than only the papers already known to disagree, specifically to check whether a fix aimed at one construct could have broken an already-correct answer elsewhere. Six of the seven had not; the seventh — the R-package-source-tree exclusion, meant to drop a bundled dependency's own scaffolding (`DESCRIPTION`, `NAMESPACE`, `R/`, `man/`, `tests/`) from a repository's file listing — also stripped a paper's own analysis code whenever that code was itself packaged as an installable R package, a common practice for ecological modelling and analysis tools shared alongside a methods paper, and in several cases stripped the package's bundled `data/` directory as well, turning a real dataset into an apparent absence of one. This was confirmed against 16 real GitHub/CRAN-hosted repositories (`RevGadgets`, `BaBA`, `m2b`, `RRPP`, `actel`, `scampr`, `MINOTAUR`, `LadderFuelsR`, `biomonitorR`, `EcoEnsemble`, `fossilsim`, `frair`, `tRophicPosition`, `sabinaNSDM`, `bef`, and `phylomice`). Rather than being left as an open limitation, the exclusion rule was corrected within this same review cycle: it now only strips a root-level package's files when nothing classifiable as real data or code survives outside it — using the same `data_classify_files()` rules-based classifier `data_check` itself relies on — distinguishing a study that shares its own data and code as an R package (nothing survives outside it, because the package is the deposit) from a repository that bundles a vendored copy of someone else's package alongside the study's own files (real content survives outside the vendored package, so only the vendored package's files are excluded). Reapplying the corrected rule resolved 11 of these 16 cases; the remaining 5 are due to an unrelated, still-open gap where the affected repository's own cached download never retrieved its `R/` source directory in the first place, so no exclusion-logic fix can recover content that was never fetched.

We see this kind of construct-by-construct, evidence-grounded mapping — quantifying not just how often Metacheck and a trained human coder disagree, but why, down to the individual

paper and, where applicable, the specific line of code responsible, reverifying that mapping against a real fix rather than stopping at diagnosis, and checking a fix’s effect on the papers it was not meant to change as carefully as its effect on the papers it was — as the more useful contribution here than a single aggregate accuracy figure, both for meta-scientists deciding whether Metacheck can currently support a given coding task, and for Metacheck’s own development priorities. Appendix A documents, in full detail, every defect identified over the course of this project, the evidence that surfaced it, the exact code change that fixed it, and the before/after numbers at each stage; the body of this manuscript reports only the current, post-fix numbers.

Detailed record of every Metacheck defect found and fixed during this project

This appendix documents, in full detail and in chronological order, every software defect in Metacheck that this project’s validation effort identified, the evidence that surfaced each one, the exact code change that fixed it, and the corpus-level numbers before and after that fix. The body of the manuscript reports only the final, current numbers; this appendix exists so that every intermediate claim made in the course of arriving at those numbers remains checkable, without requiring the main text to carry a running commentary on its own revision history. Seven rounds of validation are documented below: (A.1) the original empirical-comparison review against Cooper et al.’s coding, which produced issues #378–#384; (A.2) a focused re-examination of `code_check`’s parse-error and package-version-pinning checks; (A.3) a systematic validation of five further `code_check` checks (missing referenced files, hardcoded absolute paths, `setwd()` calls, comment density, and library/import scattering) against real corpus files, which produced five further fixes; (A.4) a file-count gate added after one outlier repository’s processing time exposed a gap the existing byte-size caps did not cover; (A.5) a full re-check of every remaining disagreement against Metacheck after five more fixes (Zenodo reliability, repository-platform coverage, and a rate-limit reliability defect found live during the re-check itself); (A.6) a systemic under-sampling bug found in A.5’s own Dataverse DOI-prefix allowlist, two further platform-coverage fixes (DataONE, institutional Figshare instances) and two matching bugs in this project’s own comparison code, and a complete, evidence-based categorization of every “repository not detected” disagreement these fixes did not resolve; and (A.7) a full-corpus run of Metacheck’s `reproducibility_check` module — a different question from A.1–A.6 (does a paper’s own shared code actually run and reproduce its own reported results, not merely whether a repository was detected) — which surfaced five further defects specific to running many papers’ Docker containers concurrently at corpus scale and to extracting a paper’s own declared dependencies from its code. Fixes described in A.1–A.4 were committed to `scienceverse/metacheck`, branch `repocheck_improvements` (pull request #385); A.5 and A.6’s fixes were committed directly to that package’s `dev` branch (A.6 as pull requests #410, #412, and #414); A.7’s five defects were filed as issues #417–#421, two of which (#419, #420) are confirmed fixed on `dev` as of this writing. Every fix from A.1–A.6 is covered by a new or updated automated regression test in the package’s own test suite; A.7’s two confirmed-fixed defects were fixed and committed by the package’s maintainer rather than by this project directly, so this manuscript does not independently confirm test coverage for them.

A.1 Defects found during the original empirical comparison

Six defects were identified while individually reviewing every disagreement between Metacheck and Cooper et al.’s human coding (Empirical Comparison, above): five in Metacheck itself, and one in this project’s own comparison code. All six are described here with the same level of

detail as in the body text; they are repeated in this appendix only for completeness, so that Appendix A is a single, complete record of every fix made during this project.

#378 — `zip_peek()`'s unauthenticated requests. `zip_peek()` (`R/zip-peek.R`) reads a remote zip archive's central directory via an HTTP Range request, without downloading the archive, so a downloader can decide whether the archive is worth fetching before spending the bandwidth. This function's HEAD and Range requests were unauthenticated. Dryad's file-download endpoint answers 401 Unauthorized to any request that touches file bytes, including a bare HTTP Range request — confirmed live against a real Dryad file-download URL — so every Dryad-hosted archive's peek failed structurally, before any question of the archive's actual contents. Fix: both requests now route through `.auth_for_url()`, the same host-based authentication dispatcher every other Metacheck download path already uses, so Dryad's token is attached automatically without requiring each call site to opt in individually (an earlier, per-call-site `req_func` opt-in mechanism had only been adopted by 1 of 7 call sites that needed it).

#379 — `data_check`'s multi-sheet Excel limitation. `data_check`'s tabular-usability check called `readxl::read_excel()` with no `sheet` argument, so it only ever read a workbook's first sheet. When the first sheet was a cover, metadata, or “read me” sheet — a convention confirmed across many real corpus files, with sheet names including “metadata,” “Meta-data,” “MetaData,” “read me,” “ReadMe,” “User guide,” “LEGEND,” and “Description of variables” — a later sheet holding genuine, well-structured tabular data was never inspected, and the file was misclassified as unusable (“100% free text, not variables”). One confirmed case: sheet 2 of a real downloaded workbook held 1,532 rows by 58 real variable columns, never read before the fix. Fix: `data_read_head()` now accepts a `sheet` argument, and `data_check` tries each remaining sheet in a workbook before giving up on the file. This was the single largest driver of `data_download` disagreement: before the fix, `mc_data_download` and Cooper et al.'s `data_download` disagreed on 51 of 967 comparable papers (94.7% agreement); after it, agreement rose to 99.1% (908/916 papers, 8 disagreements remaining), and the resolved cases accounted for 29 of the original 51 disagreements (57%).

#380 — Dryad's authentication gap on other download paths. The same unauthenticated-request problem identified for `zip_peek()` (#378) also affected `.remote_size()` and `.remote_content_length()`, two other functions in the download path that check a remote file's size via HTTP HEAD before deciding whether and how to download it. `.remote_content_length()` already had a `req_func` opt-in mechanism for per-host authentication, but only 1 of 7 call sites across the codebase had actually adopted it, so the other 6 still received a 401 from Dryad. Fix: both functions now route through `.auth_for_url()` unconditionally, the same fix applied to #378, closing every remaining unauthenticated-request gap in the download path at once rather than patching call sites one at a time.

#383 — `code_check` could not see code bundled inside a zip archive. This was a compound architectural gap with three separate causes, discovered while investigating why `code_check` found no code files at all for several papers whose repository, on manual

inspection, clearly contained real analysis scripts bundled inside a `.zip`: (1) `code_check()` never read `data_check`'s already-expanded file structure, so any code file `data_check` had already discovered inside a zip (in the course of looking for data files) was invisible to `code_check` regardless; (2) `code_check` had no zip-peek capability of its own for the case where `data_check` had not already run first; (3) the shared zip-expansion helper `.archive_rows()` discarded code-typed files outright even when it did expand a zip, keeping only `"data"` and `"documentation"`-typed files. Fix: `code_check()` now prefers `data_check`'s structure when available; it gets its own zip-peek fallback (`.code_expand_zip()`) for when `data_check` did not run first; and `.archive_rows()` now keeps `"code"` alongside `"data"/"documentation"`. A real, concrete example verified live: a Zenodo record (record 3694205, repository SharpRT/CMV_Spread) bundled 4 real MATLAB `.m` files inside a zip that were entirely invisible to `code_check` before this fix. This fix, together with the archive-member content-fetch wiring described immediately below, moved `code_download` agreement from 66.2% (355 comparable papers, 120 disagreements) to 89.7% (358 comparable papers, 37 disagreements) — the first of two stages in this construct's improvement (the second is #384, below).

As part of the same round of work, `repo_check`'s zip-peek expansion was extended to carry not just a discovered code file's name (already possible after the fixes above) but its content: each expanded row now also carries `archive_url` (the parent zip's own URL) and `archive_member` (the file's exact name inside it), and `download_repo_files()` (R/`repo-download.R`) uses those two fields to fetch that one member's bytes directly, via a byte-range request against the parent archive, without downloading the archive's other contents. This mechanism was verified working in every isolated unit test at the time, but a full end-to-end pipeline run against a real paper still returned no file content for reasons not yet diagnosed at that point — this was filed and tracked as issue #384, described next, rather than merged as “fixed” while a known gap remained open.

#384 — Two distinct causes behind the archive-member byte-fetch failing intermittently. Issue #384 was filed as an open question (“archive-member content fetch works in isolation but returned NA once in a full pipeline run”) rather than a diagnosed bug, and its investigation spanned two separate validation efforts and uncovered two independent causes.

Cause 1: silent truncation over 32,768 bytes. `.zip_inflate_member()` (R/`zip-peek.R`) decompresses one zip member's raw bytes by calling `zip::inflate()`. Passing the member's exact known uncompressed size to that function raised an unrelated “Failed to inflate data” error (a quirk internal to the `zip` package), so the original code left the `size` argument unset instead. `zip::inflate()`'s own documentation states that leaving `size` unset triggers automatic buffer resizing as needed — but, confirmed live against a real Zenodo-hosted zip member, this documented behaviour does not actually happen in practice: any member larger than the default 32,768-byte output buffer came back silently truncated at exactly that length, with no error, no warning, and a `bytes_written` field the calling code never inspected. A concrete confirmed case: a real 44,669-byte code file came back as exactly 32,768 bytes. `.zip_member_fetch()`'s own subsequent length check (comparing the decompressed length against the size recorded in the archive's own central directory) correctly caught this mismatch and discarded the truncated

bytes — but the practical, user-visible effect was that every code or data file over 32KB inside a zip silently failed to fetch, indistinguishable from a genuine network or host-compatibility failure. Fix: passing `size + 1` (one byte over the true size) avoids both failure modes — it is no longer the exact size that triggers the package’s internal error, and it is large enough that the real output always fits in the up-front buffer rather than depending on the documented-but-non-functional resize path. Verified against both a real archive member under 32KB and one over it, and covered by a new offline regression test using a locally constructed zip file whose one member compresses to roughly 95KB uncompressed. A targeted rerun of the 36 papers affected by this pattern found 27 moved from zero downloadable code files to full or near-full recovery (23 fully, 4 partially), which is what raised `code_download` agreement from 89.7% (37 disagreements) to 97.2% (10 disagreements remaining) — the second of the two stages in this construct’s improvement, and the largest single agreement-rate improvement of any construct in the empirical comparison.

Cause 2: an intermittent, silently-swallowed failure not explained by the truncation bug alone. Independently of the truncation bug, a second cause was found while investigating why 40 of the archives affected by issue #384 failed entirely (every member missing) despite containing no member anywhere near the 32,768-byte threshold. Tracing the actual corpus data (`res_code_check.RData` and its 38 per-batch constituent files, in `cooper_validation_metacheck/manuscript/data/`) showed that a single pipeline run calls `zip_peek()` on the exact same archive URL twice — once inside `repo_check` to discover a zip’s member names, and again inside `download_repo_files()`’s archive-member-fetch step (added as part of #383/#384’s own fix) to retrieve one member’s bytes — with no throttling and, critically, no retry logic at all on either call: `zip_peek()`’s own HEAD request and its two Range-request helper functions (`.http_range_tail()`, `.http_range_bytes()`) called `httr2::req_perform()` directly, with no `req_retry()`, and treated any non-206/200 HTTP status (including a 429 burst-rate-limit refusal) as an unconditional, silent NULL — indistinguishable, by design, from “this host does not support range requests at all.” Two back-to-back, unthrottled requests to the same archive URL within one process is exactly the pattern most likely to trip a host’s burst-rate limiter, and the second call (the byte-fetch, inside `download_repo_files()`) was the one exposed to it. Fix, applied in two parts: (1) all three of `zip_peek()`’s network calls now retry through the same `is_transient/backoff/rate-limit-wait` helper functions (`.storage_is_transient_factory()`, `.storage_backoff()`, `.storage_retry_after_factory()`) that every other Metacheck download path already uses, so a 429 is now retried with backoff rather than immediately and silently treated as “unsupported”; (2) a new same-session, in-memory cache (`.zip_peek_cache`, keyed by URL) means the second call to `zip_peek()` on an archive already peeked earlier in the same run reuses the first call’s result instead of repeating the network request at all, removing the redundant call — and the rate-limit exposure it caused — entirely rather than merely tolerating it. Verified by a new regression test exercising the cache directly (`test-zip-peek.R`) and by the full package test suite (463 tests passing, 3 pre-existing unrelated skips, 0 failures) after both changes.

Verdict corrections found by the post-rerun double-check pass. After the full corpus

rerun incorporating #378–#384, every disagreement was independently double-checked a second time (see Empirical Comparison, above, for the check’s method). This pass corrected 7 verdict-labeling errors from the original review: 2 in `data_availability` (one moved to `COOPER_RIGHT` after finding the “data” files actually belonged to a cited third-party software dependency’s repository, not the paper’s own; one moved to `METACHECK_RIGHT` after finding a methods paper’s repository, originally read as containing only package example files, in fact holds real trait data and model output), 1 in `data_download` (moved to `COOPER_RIGHT` after confirming a Zenodo-hosted “Functional Trait Database” file had, in fact, already downloaded and still failed the tabular-usability check, rather than remaining `UNCLEAR` on the assumption it had not yet downloaded), and 4 in `code_download` (three rows originally recorded `METACHECK_RIGHT` whose own written reasoning already concluded the opposite, corrected to `COOPER_RIGHT`; one row originally left `UNCLEAR` where the code files were, on direct inspection, confirmed genuinely downloaded, corrected to `METACHECK_RIGHT`). The pass also found one substantial scope correction rather than a single-row relabeling: `code_download`’s dominant post-#383 disagreement pattern was originally tagged in only 13 of its 37 disagreements at the time of the initial qualitative review; re-deriving the pattern systematically against every disagreement’s saved `file_location` value found the identical signature (a discovered code file with no resolvable download location) in 35 of the 37, which is what led directly to tracing and fixing #384’s second cause, above. Every one of `code_archived`’s 164 disagreements was also individually re-derived against the current `repo_check/code_check` file listing during this pass, including the 8 originally resolved via an automated keyword search rather than an individual hand read; this found zero verdict corrections in that column.

A sixth issue, not in Metacheck but in this project’s own comparison code. `code_data_preparation/03_recreate_cooper_columns.R` derives `mc_data_availability`, `mc_code_archived`, `mc_data_README`, and `mc_code_README` for a paper from `repo_check`’s per-file tables (`res_repo_check$table`, `res_data_check$structure`, `res_code_check$table`). A paper for which `repo_check` found zero files anywhere has no row at all in any of those three tables, even though `repo_check`’s own per-paper `summary_table` — which covers every one of the 1861 papers — correctly records `files_n = 0` for it. Because the script built its paper list from the three per-file tables rather than from `summary_table`, every such paper was silently absent from the comparison entirely: its `mc_*` value was not `FALSE`, it simply never existed as a row to compare. Confirmed directly: 495 of 1861 papers were missing this way, including 404 papers Cooper et al. coded `data_availability Yes` — 404 real disagreements (cases where Cooper et al.’s coders found a repository Metacheck’s own detection had missed) that were never entering the comparison at all, rather than being counted as a genuine miss. Fix: the paper list is now built from `summary_table$paper_id` (already the full 1861 by construction), and `mc_data_availability`, `mc_code_archived`, `mc_data_README`, and `mc_code_README` are explicitly set to `FALSE` for a paper `repo_check` examined and found nothing in; every other `mc_*` column (license, DOI, download, format, language) is correctly left `NA` for such a paper, since those questions remain genuinely inapplicable without a repository to ask about. Every number reported in the Empirical Comparison section above reflects this correction.

A.2 Re-examining `code_check`'s parse-error and package-version-pinning numbers

A separate line of inquiry, prompted directly by a sentence in an earlier manuscript draft stating that “138 files (1.3%) failed to parse as valid code in their declared language at all,” asked two questions: is this the right denominator, and is the file list behind it available for inspection at all.

Locating the underlying data. `code_check()`'s per-run output already carries everything needed: a `table` with one row per code file (`file_name`, `language`, `parse_error`, `parse_error_msg`) and a `summary_table` with one row per `paper` (`code_version_pinned`, `code_parse_errors`), both already computed by the module — no new code was needed to make this data available, only to locate and aggregate the 38 saved per-batch corpus-run files (`cooper_validation_metacheck/manuscript/data/batches/res_code_check_batch001.RData` through `batch038.RData`) that had already been produced by the original corpus run.

Package version pinning: 34.2% was a batch-level upper bound, not a per-paper rate. An earlier manuscript draft reported that “only 13 of 38 batches (34.2%) contain at least one paper with any form of version pinning,” explicitly flagged in that draft's own text as an upper bound on the true per-paper rate because a batch of ~50 papers needs only one pinned paper to count. Checking directly against the saved per-batch `summary_table$code_version_pinned` column (present in every batch, keyed by `paper_id`) confirmed the per-paper detail this draft believed was not retained had, in fact, been retained all along: aggregating it across all 38 batches gives 18 of 400 code-bearing papers (4.5%) with version pinning present — substantially lower than the 34.2% upper bound, as expected, and now a true per-paper number rather than an aggregate-of-aggregates. No code fix was needed here; this was a reporting correction (using data that already existed but had not yet been extracted at the right level of granularity), not a software defect.

Parse errors: 1.3% used the wrong denominator, and undercounted for a subtler reason. The originally reported 1.3% divided 138 parse failures by 10,461, the total code-file count across every language in the corpus — but parsing is only ever attempted on R-type files (`.R`, `.Rmd`, `.qmd`; `code_parse_r()` is R-only), so the other 3,259 SAS/SPSS/Stata/Python/MATLAB files in the corpus were never eligible to fail this check and should not have been in the denominator at all. Restricting to the corpus's 7,202 R-type files, a further complication emerged: 1,651 of those were never actually downloaded or read at all (an inaccessible repository or a failed download), so parsing was never attempted on them either — they, too, do not belong in the denominator. A further 15 files reached a local copy but errored during extraction (before `parse()` itself was ever called), splitting into two genuinely different causes on closer inspection: 11 were confirmed to be genuinely empty (0-byte) files — fetched directly from their original GitHub repositories to confirm the emptiness is a fact about the authors' own repository, not an artefact of Metacheck's download step (e.g. github.com/olgaviedma/LadderFuelsR's `R/foofy.R` and github.com/dpritchard/frair's `tests/testthat/test_emdII.R`, both independently confirmed 0 bytes via a direct `curl` fetch from `raw.githubusercontent.com`) — and 3 were `.Rmd` files that failed during Quarto/knitr's own chunk-extraction step with

a genuine, structural fence-mismatch error (a closing code-fence with a different number of backticks or indentation than its opening fence), which is the same underlying “not valid in its declared format” problem a parse error represents, just caught one step earlier in the pipeline than `parse()` itself. Restricting to the 5,536 files that were both locally available and actually reached a call to `parse()`, 138 failed (2.5%) — not 1.3%, and not the 1.92% that a denominator of “all 7,202 R-type files regardless of whether they were ever read” would give either. Including the 3 fence-mismatch failures as the same kind of problem gives 141 of 5,539 (2.5%).

This investigation also surfaced a genuine, previously unknown software defect: **`code_read()` (R/`code_check.R`) crashes on a genuinely empty file.** `readr::guess_encoding()` returns a zero-row result for a 0-byte file, so `enc <- readr::guess_encoding(file_path)$encoding[[1]]` silently produces NA rather than a valid encoding string, and that NA then crashes deep inside `readr`’s own internals (`if (stringi::stri_enc_isascii(lines))`), which requires a length-one logical and receives NA instead) with the unhelpful, generic message “missing value where TRUE/FALSE needed” — this is exactly the message recorded for all 11 of the empty-file cases above, rather than a clear “this file is empty” signal. This defect is documented here as confirmed and diagnosed, but **not yet fixed** as of this writing (it affects a small, already-quantified fraction of files — 11 of 7,202 R-type files in this corpus — and a fix has not yet been prioritized ahead of the five `code_check` checks fixed in A.3 below); it is recorded here so the fix, when made, has a clear before-state to compare against.

A.3 Systematic validation of five further `code_check` checks

A third, more systematic validation exercise checked five of `code_check`’s remaining automated checks — missing referenced files, hardcoded absolute paths, `setwd()` calls, comment density, and scattered library/import lines — against real files from the same corpus, reading every flagged file that had already been downloaded locally (8,218 of 10,461 code-file rows) and sampling up to 100 of the remainder that would have required a fresh download. The full method, every sampled file’s judgment, and every quoted example are recorded in a dedicated, reproducible validation report (`code_check_validation.qmd`, with its accompanying script `validate_code_check.R` and judgment tables, all in `cooper_validation_metacheck/manuscript/`) with a fixed random seed (`set.seed(42)`) at every sampling step; this appendix summarizes its findings and the resulting code fixes. Two checks not covered by this validation pass (parse errors, package version pinning) are covered separately in A.2 above.

Missing referenced files — case-sensitivity bug (fixed). `code_check`’s check for files a script tries to open but that are absent from the repository (`inst/modules/code_check.R`, `missing_files <- setdiff(base_ref, base_repo)`) compared filenames case-sensitively. 13 confirmed false positives across the sampled files were all the same mechanism: the referenced file exists in the repository under a different letter case, which is not a real error on the author’s part on the case-insensitive filesystems (Windows, macOS by default) most authors develop on. Concrete examples, each confirmed directly against the paper’s real repository file listing: paper

10_1002_2688_8319_12229 references `REM_tools.r` where the repository holds `REM_tools.R`; paper 10_1111_2041_210x_13923 references `diagnostic_fcns.r/boot_glmm.r` where the repository holds `diagnostic_fcns.R/boot_glmm.R`; paper 10_1111_2041_210x_13125 references `FoodWeb_Functions.R` where the repository holds `Foodweb_functions.R`; paper 10_1002_pan3_10736 references two scripts whose repository copies differ only in the first letter's case. Fix: the comparison now lowercases both sides before comparing (`missing_files <- base_ref[!tolower(base_ref) %in% tolower(base_repo)]`), removing all 13 confirmed false positives found in this sample and any of the same kind not yet sampled.

Missing referenced files — `vroom::vroom()` not recognised as a file-reading call (fixed). The same check's file-reference extractor (`code_file_refs()`, `R/code_check.R`) recognises a fixed list of R reader-function names (`read.*/read_*`, `import`, `fread`, `readRDS`, `load`, `readLines`, `fromJSON`, `readtext`, `source`); `vroom::vroom()` does not begin with “read” and was absent from this list, so a file loaded exclusively via `vroom::vroom()` was invisible to the missing-file check regardless of whether it actually existed in the repository. Confirmed false negative: paper 10_1111_2041_210x_13440's `R/ncbi.R` calls `vroom::vroom(file.path(dir, "nodes.dmp"), ...)` and a second `vroom::vroom()` call for `names.dmp`; neither file exists in the repository, but the check never flagged either, since `vroom` was not on its keyword list. Fix: “vroom” added to the reader-keyword list.

Missing referenced files — remaining known limitations, not yet fixed. Two further false-negative mechanisms were confirmed but not fixed, since each would require a more substantial change than a keyword addition or a comparison tweak: (1) `code_file_refs()` only captures a quoted filename appearing on the same line as a reader-call keyword, so a file assigned to a variable on one line and read via that variable on a later line (`liz_file <- "example_lizard_data.csv"`, then later `read.csv(liz_file)`) is invisible to the check even when the file genuinely does not exist (confirmed: paper 10_1111_1365_2435_13993's `tests/testthat/test-m_run_biophysical.R` does exactly this for three files, none of which exist in the repository). (2) When a referenced filename is built as `paste0(some_variable, "...literal_suffix.csv")`, the check's regex only captures the literal trailing fragment (e.g. `_M_D_abundance_interactions.csv`) rather than the true, variable-interpolated filename — the resulting “missing file” flag is still technically correct (nothing by that literal fragment name exists either), but the name shown to a paper's author is a confusing partial match rather than the file they would recognise; across the corpus, 128 of 2,599 (4.9%) genuinely-missing entries have a name starting with punctuation, the signature of this dropped-prefix pattern.

Hardcoded absolute paths — three false-positive mechanisms fixed, one left open. `code_abs_path()` (`R/code_check.R`) regex-matches quoted strings shaped like an absolute filesystem path. Three confirmed false-positive mechanisms were fixed: (1) a single-segment string beginning with `/` (e.g. `self.fetch('/login')` in a vendored web-app test suite, `github.com_ccrisan_motioneye/tests/test_handlers/test_login.py`) is an HTTP route, not a filesystem path — 322 of 4,182 originally matched strings were 6 characters or fewer, a class dominated by exactly this pattern; the regex's bare-`/`-prefixed branch now requires a second `/`-separated segment before matching, which a real absolute

path (e.g. `/Users/name`) always has and a bare route never does. (2) An all-digit UNC “host” is a `sub()/gsub()` regex backreference string, not a network path — confirmed twice in real code: `osf.io_5fter/IDcallR_sameN_withIDcallPP.R` line 183, `sub('group(\\d+)_(\\d+)', '\\2\\1', colnames(clupo))`, where “`\\2\\1`” was matched as “`\\host\\share`” because a bare digit satisfied the old hostname character class; and a second, independent instance in a Pyrenees-mapping script’s `gsub('(.{1,10})(\\s|$)', '\\1\\n', ...)` call, matched for the same reason. The UNC branch’s hostname character class now requires at least one letter, which excludes both confirmed cases without excluding any real hostname (no genuine UNC host is all-digit-and-punctuation with no letter at all). (3) The pattern’s own pre-existing docstring already flagged a known risk — a variable-built path like `paste0(dir, '/file.csv')` triggering a false match on the literal fragment — and this was confirmed for real: `github.com_YvesBas_Tadarida-C/Sonotypes/write_tabase3HF_Sonotype.R` line 40, `list.files(paste(ListDate[[i]], "/eti", sep=""), ...)`, where “`/eti`” is a relative fragment concatenated onto a loop variable. This third mechanism is addressed by the same “require a second path segment” fix as (1), since `/eti` is itself single-segment. A fourth mechanism, confirmed but **not fixed**: a Python string built by `+`-concatenation from a relative literal (`'data\\' + tile_name + '\\run\\' + model_name + ...`, from a Zenodo-hosted `model_test.py`) is a relative path whose middle segments (`\\run\\`, `\\output\\tile\\`) still match the UNC branch on their own; no fully static regex-only fix exists for this case, since it would require tracking whether the enclosing expression is a string concatenation starting with a relative literal, which is beyond what a line-based regex can determine. Of the true positives sampled for comparison, 15 of 15 further-checked “real username-shaped” matches (e.g. `C:/Users/reddin/Documents/...`, `/Users/farrer/Dropbox/...`, `/home/michael/Dropbox/...`) were confirmed genuine, machine-specific absolute paths — the large majority of this check’s flags are correct.

setwd() calls — no false positives found in this corpus; one confirmed-but-unobserved risk noted. All 194 distinct `setwd()` call strings found across the sampled files are genuine, live, executable calls, including calls whose argument is itself relative or dynamic (`setwd("./data")`, `setwd(dir)`, `setwd(tempdir())`) — these correctly count as flags under this check’s own stated rule, since the anti-pattern being flagged is changing the working directory at all, not specifically a hardcoded absolute-path argument. A latent false-positive mechanism was confirmed by construction (a synthetic test, not a real corpus case): `code_setwd()` has no awareness of string-literal context, so a line like `message("Please run setwd(your_path) before continuing")` would be matched as if it were a live call. Scanning all 194 real call strings in the corpus for instructional-text markers (“please,” “before,” “instructions,” “note:”) found zero real occurrences of this pattern — a real, demonstrated risk with no confirmed instance in this corpus, and left unfixed for that reason, though `code_setwd()` could straightforwardly reuse the same quote-context-tracking logic already implemented for `.code_strip_inline_comment()` if a future corpus surfaces a real case.

Comment density — a Python-specific docstring signal added (not a bug fix). `code_remove_comments()`’s Python branch deliberately never treats a triple-quoted string as a comment, since doing so would also destroy real docstrings (themselves ordinary string

literals, syntactically). The consequence, quantified directly: of 280 zero-comment-flagged Python files in the corpus, 200 (71.4%) contain at least one complete triple-quoted docstring block — real documentation that the mechanical `#`-only comment count cannot see, and so reports as “0% comments” regardless. Five were read directly to confirm these are genuine module/function docstrings rather than incidental triple-quoted data, e.g. `model_runs.py`’s `"""Filters for Model Runs."""` and `features.py`’s 16 separate triple-quote markers (8 docstrings) opening with `"""API functions to interact with feature names."""`. This is a deliberate, correctly-reasoned design choice in `code_remove_comments()`, not a bug — stripping docstrings to count them as comments would also delete them from every downstream check that reads code text — so the fix made is additive rather than corrective: a new function, `.code_has_docstring()`, detects whether a file contains at least one complete triple-quoted block, and `code_line_stats()` now returns this as a separate `has_docstring` field (NA for every non-Python language) alongside the unchanged `percent_comments` metric. `inst/modules/code_check.R`’s report text now adds an explanatory note whenever a zero-comment Python file in fact has a docstring, so a reader is told the file is documented via docstrings rather than concluding it has no documentation at all.

Scattered library/import lines — Python import-anchor tightened (fixed). `code_library_lines()` (`R/code_check.R`) previously anchored its Python import-detection regex on the bare keyword plus a following word (`^\s*(import|from)\s+[A-Za-z_.]`), with no requirement that the rest of the line actually form a valid import statement. Because Python’s comment-removal never strips triple-quoted strings (the same mechanism behind the comment-density finding immediately above), docstring prose that happens to start with “from” or “import” survives into this scan and was matched as if it were a real import line. Concrete, exact example: `github.com/opencv/opencv4/site/process_sdk_docs.py`, line 213, inside a docstring spanning lines 211–214 — the prose sentence “from the generated model and api descriptions.” was matched as an import line at line 154 of the comment-stripped text, while the file’s 10 real imports are all genuinely contiguous at lines 1–10; the spurious match alone widened the reported gap between imports to 144 lines, crossing the “scattered” threshold (more than 3 non-comment lines between library statements) on a file whose real imports are, in fact, perfectly grouped. Quantified across all 525 Python files flagged as scattered in the corpus: 155 (29.5%) have at least one spurious, non-import match of this kind; of those, 68 (13.0% of all 525 flagged files) would no longer be flagged as scattered at all once the spurious matches are excluded — pure false positives caused entirely by this mechanism (the remaining 87 files with a spurious match are still genuinely scattered even after excluding it). R showed essentially no equivalent problem in a 60-file sample (the one apparent candidate, on manual inspection, was a genuine conditional `require()` call, not a false positive — R’s regex requires an opening parenthesis directly after the keyword, structurally harder to spuriously trigger than Python’s bare whitespace-plus-word anchor). Fix: the Python pattern now requires the full import shape rather than just the leading keyword — a `from` line must also contain its own `import` keyword later on the same line (`^\s*from\s+[A-Za-z_][A-Za-z0-9_.]*\s+import\s`), and a bare `import` line must be followed by a comma-separated identifier list with an optional `as name` and nothing else (`^\s*import\s+[A-Za-z_][A-Za-z0-9_.]*(\s*,\s*[A-Za-z_][A-`

`Za-z0-9_.]*)*(\s+as\s+\w+)?\s*$)` — verified against the docstring false-positive example above (now correctly unmatched) and against every real import shape found in the corpus sample, including multi-line `from x import (a, b, c)` continuations (matched on their opening line) and indented imports, with no loss of any real detection.

Status of the fixes in this appendix. All Metacheck fixes described in A.1 and A.3 (nine in total: #378, #379, #380, #383, both causes of #384, and the five `code_check` fixes from the systematic validation) have been committed to `scienceverse/metacheck`’s `repocheck_improvements` branch (pull request #385) and are covered by the package’s automated test suite (463 tests passing at time of writing, 0 failures, 3 pre-existing skips unrelated to these changes). The sixth fix described above (the comparison script’s missing-repository default) is not a Metacheck fix and has been committed instead to this project’s own `code_data_preparation/03_recreate_cooper_columns.R`, in the `cooper_validation_metacheck` repository. The one defect identified but not yet fixed (`code_read()`’s crash on an empty file, A.2) and the two `code_file_refs()` limitations noted but not yet fixed (A.3) remain open. The further full corpus rerun incorporating the five A.3 fixes, planned when this appendix was first written, has since been performed (confirmed directly: the current corpus run’s `code_check` output carries the `has_docstring` field A.3’s docstring fix introduces, which cannot exist in a pre-fix run) — the numbers reported in “Results you get for free alongside Metacheck” above reflect that post-fix corpus, not the pre-fix state described earlier in this sentence’s original drafting. This was a distinct item from the rerun described in Appendix A.5 below, which re-checked only the papers with a remaining disagreement (not the full corpus) and tested a different set of fixes (Zenodo reliability, repository-platform coverage, and rate-limit handling), not the five A.3 `code_check` fixes.

A.4 A repository excluded from processing by file count

During the corpus rerun incorporating the A.3 fixes, one paper’s data repository — Zenodo record 10.5281/zenodo.3971689, SAPFLUXNET (a global database of sap flow measurements: 202 datasets from 121 locations, sap flow and environmental time series for 2,714 individual plants) — was found to hold approximately 4,000 individual files. Downloading and, in particular, `data_check`’s per-file classification and full-content read of every one of those files (`data_check_head(..., n_rows = Inf)`, by design, since several of `data_check`’s validation checks need the complete dataset rather than a sample) took several hours and multiple gigabytes of memory for this single paper, out of 1,861 in the corpus, before the run was stopped.

This exposed a real gap in Metacheck: `download_repo_files()` already gates a repository outright by total byte size (`max_download_size`) or per-file byte size (`max_file_size`), but had no gate on file count — a repository holding thousands of individually small files was not caught by either existing cap. A new parameter, `max_files_per_repo` (`R/repo-download.R`, threaded through `data_check()` and `code_check()`), gates a repository by file count alone,

using the same refusal/reporting mechanism as the existing size caps. This has been committed to the `repocheck_improvements` branch alongside a regression test, and the corpus rerun was configured with `max_files_per_repo = 1000`.

Under this cap, the SAPFLUXNET repository is refused outright rather than downloaded, so this one paper’s `data_check/code_check` results are incomplete (no files analysed for it) in the corpus run reported in this manuscript. This is a deliberate, scale-driven exclusion of a single outlier repository, not a defect in how that paper’s data was assessed — the cap exists specifically so that one exceptionally large repository cannot block or degrade processing of the other 1,860 papers in the corpus. Revisiting this one paper’s repository with a raised or absent cap, as a separate, deliberate pass, remains an open option.

A.5 A targeted rerun on every remaining disagreement, after further platform-detection and reliability fixes

After Appendix A.1–A.4’s fixes and the resulting full corpus rerun, every remaining disagreement between Metacheck and Cooper et al.’s coding (991 rows across 989 papers, the same set individually adjudicated for the Empirical Comparison above) was re-checked a further time against Metacheck as it stood after seven more commits landed on `scienceverse/metacheck`’s `dev` branch: two fixing crashes and false negatives in Metacheck’s Zenodo integration, three widening which repository platforms `repo_check` can locate a repository on at all (targeting “repository not detected,” the single largest recurring disagreement cause identified in the original review — see “The five recurring causes” above), and two fixing a rate-limit reliability defect surfaced live during this rerun itself. Rather than reprocess the full 1,861-paper corpus, only the 989 papers with at least one open disagreement were rerun, one paper at a time (so that a single paper’s slow host response could not block any other), reusing the shared on-disk repository-file cache from the original corpus run so that a repository already fetched and unchanged cost no new download.

Zenodo reliability. Two fixes addressed real crashes and false negatives in Metacheck’s Zenodo integration, confirmed against Zenodo records already present in this corpus. `online()`, the function `repo_check` calls to confirm a host is reachable before querying it, treated a single transient DNS lookup miss as “Zenodo is offline” and abandoned that paper’s entire Zenodo retrieval rather than retrying; it now retries. Separately, a JSON metadata field arriving as a length-zero value (an empty array) rather than `NULL` defeated the `%||%` operator used throughout Metacheck’s metadata-extraction code (which only guards against `NULL`, not length-zero), producing a `data.frame()` “arguments imply differing number of rows” error that was silently swallowed inside a `tryCatch`, discarding the whole record; a new `%empty_or%` operator, applied everywhere the same gap existed (Dataverse, Dryad, DSpace7, Figshare, GitHub, GitLab, OSF, PsychArchives, and ReShare, not just Zenodo), closes this. A further, closely related pair of crashes was found and fixed the same way: a Zenodo record Metacheck could not read at all previously returned a `data.frame()` with only `zenodo_id/error` columns, so a paper whose every Zenodo record was unreadable produced a metadata table with no

`doi/license` columns at all, which crashed `repo_check()`'s own Zenodo-metadata-extraction step for that paper the same “differing number of rows” way; and a `repo_url` arriving as a list column (rather than a plain character vector) could silently explode `data.frame(zenodo_url = <list>)` into one column per list element instead of the expected single column. Both are now guarded.

Repository platform coverage. Three commits substantially widened the set of hosts Metacheck's `repo_check` can locate a repository on at all, directly targeting the “repository not detected” cause responsible for the largest share of the original disagreement (see “The five recurring causes” above, which named DataSTORRE, DIGITAL.CSIC, AADC, SRA/GenBank, KNB, Neotoma, CIRAD Dataverse, and ForestPlots.net as example platforms with no backend at the time). Mendeley Data (detected via `10.17632/` DOIs and `data.mendeley.com` URLs, queried through the public, unauthenticated `data.mendeley.com/public-api/datasets/<id>` endpoint) and DataONE (a federation of independently-run repositories rather than one hosted service; four member nodes verified live and wired in — Arctic Data Center, KNB, Taiwan Forestry Research Institute, and the Smithsonian Tropical Data Portal, each mounting the same underlying REST API at a different base path) were added as new platforms entirely. Separately, Dataverse's recognised-installation list was expanded from 50 to 121 hosts, 16 institutional Figshare white-label front ends were added, and DSpace support was generalized from PsychArchives specifically to any host running the same legacy (6) API (5 more verified live) plus a new implementation covering 64 DSpace 7 installations (e.g. Cornell's eCommons, Georgia Tech's Digital Repository, ETH Zurich's Research Collection) — every one of these $121 + 16 + 5 + 64$ hosts confirmed live via its own version/info endpoint before being added, not merely listed from a registry. A further, smaller fix excludes an R package's own source tree (`DESCRIPTION/NAMESPACE/its R/man//tests/` subdirectories) from a repository's data/code file listing, on the reasoning that reusable software infrastructure bundled alongside a study's own materials is not itself the study's data or analysis code and should not inflate `files_n/files_code` or appear in downstream reports.

A reliability defect found live during this rerun. While this rerun was in progress, a single paper stalled repeatedly at the same point across two separate attempts: `zip_peek()`'s HEAD request and its Range-request follow-up (`.http_range_tail()`, `R/zip-peek.R`), used to read a remote zip archive's file listing without downloading it, each independently hit Zenodo's rate limit and each, independently, paid the full announced wait for the host's confirmed reset — three consecutive confirmed-rate-limit waits back to back for what was, from Zenodo's own side, a single rate-limit window, since neither request had any way to know the other had already discovered and waited out the same limit. (The rate-limit-wait message itself was also, at the same time, made to name the specific host responsible — Zenodo, Dryad, GitHub, or GitLab, the only hosts that expose a machine-readable reset time — rather than reporting only “the host's own reset,” a smaller, secondary fix made in the course of diagnosing this one.) Fix: a session-scoped record of the last confirmed rate-limit reset per host, written the first time any request discovers one and consulted by every subsequent request to that host before it is even sent, so a host already known to be rate-limited waits out only the genuinely remaining time once, rather than every independent request rediscovering the identical window

from a fresh 429. Verified with new regression tests covering the record/lookup logic directly (round-tripping a wait, keeping the later of two recorded resets, clearing an expired window, and the pre-flight wait actually sleeping the correct duration) and confirmed live: the same paper that previously stalled for over 40 minutes across three stacked waits resolved in roughly 3 minutes with zero confirmed-rate-limit waits after the fix, the remaining time attributable to Zenodo’s own server-side response latency for that specific record (confirmed independently: a single isolated request to the same record, made outside Metacheck entirely, itself took 30 seconds and returned a 504 on a second attempt).

Results of the rerun. Across the 1,532 individual construct-level disagreements re-checked (989 papers $\times$ up to 7 constructs each, matching however many constructs a given paper had originally disagreed on), 76.8% (1,176 of 1,532) are now resolved — Metacheck’s fresh result agrees with Cooper et al.’s original human coding where it did not before — with 348 still disagreeing and 1 now returning a value that could not be classified as agreeing or disagreeing (`data_license`, a repository whose licence field this rerun could not read for reasons not yet diagnosed). By construct: `any_readme` 548/625 resolved (87.7%), `data_availability` 339/424 (80.0%), `code_archived` 252/360 (70.0%), `data_archive` 12/56 (21.4%, evaluated by set membership — a paper’s platform coded by Cooper now appearing among Metacheck’s own, possibly multi-platform, detected set, since a paper can genuinely link more than one repository), `data_license` 16/57 (28.1%), `data_download` 9/9 (100%), and `code_download` 0/1 (the sole remaining `code_download` disagreement in this corpus). The comparatively modest improvement in `data_archive` and `data_license` is consistent with the original review’s own finding that these two constructs’ disagreement was, unlike `data_availability/code_archived/any_readme`, not dominated by the repository-detection cause these fixes targeted (`data_license`’s largest cause was, per “Opposite direction” above, cases where Metacheck was already right and Cooper’s own coding had missed a real licence field) — a fix aimed at finding more repositories naturally does less for a construct whose disagreement was rarely about a missing repository in the first place. This rerun’s full row-level detail (every paper, every re-checked construct, the value before and after, and whether it resolved) is recorded in `cooper_validation_metacheck/manuscript/data/rerun_disagreements_comparison.csv`, produced by `rerun_disagreements.R`; the underlying per-paper `repo_check/data_check/code_check` results are cached per paper in `data/rerun_batches/` for inspection or a further rerun without recomputing what has not changed.

Checking the rest of the corpus for regressions. This rerun was scoped to the 989 previously-disagreeing papers only — a deliberate choice at the time, to get a fast answer to “did these fixes help” without the cost of a full 1,861-paper rerun (see the rerun’s own header comment for the reasoning). That scoping left one real question open: none of these seven fixes were written to only affect a disagreeing paper, so it remained possible that a fix which resolved one paper’s disagreement had, in a different paper that already agreed with Cooper et al., turned a correct result into an incorrect one — most plausibly the R-package-source-tree exclusion (if a paper’s own analysis code happens to be organised as an R package) or the expanded Dataverse/Figshare/DSpace/Mendeley/DataONE host recognition

(a newly-recognised host matching something that should not have counted). This was checked directly: the remaining 868 papers (every paper not in the 989-paper disagreement set) were also rerun, using the identical per-paper, resumable approach, and the full corpus's `cooper_vs_recreated.RData/disagreement` worklist were then rebuilt from the combined result (all 1,861 papers) rather than just the 989. At the level of aggregate agreement rates, this full-corpus check was reassuring: the three constructs this comparison's own disagreement was concentrated in — `data_availability`, `code_archived`, `any_readme` — all showed a higher, not lower, agreement rate once the previously-agreeing papers were included, and `data_download/code_download` (both already above 99%) moved by less than a percentage point in either direction. But an aggregate rate rising is consistent with a handful of individual papers having genuinely regressed, provided enough other papers improved to more than offset them — and that is exactly what the case-by-case review below found.

A confirmed regression, found and fixed within this same cycle. The suspicion named above turned out to be correct: the R-package-source-tree exclusion did break papers it should not have touched. The exclusion was written to detect an R package's fingerprint (a `DESCRIPTION` and `NAMESPACE` file in the same folder) and drop that folder's contents — the package's `R/`, `man/`, `tests/`, `vignettes/`, `data/`, `inst/`, and `src/` subdirectories — from a repository's file listing entirely, on the reasoning that a bundled, reusable software dependency is not itself a study's data or analysis code. That reasoning holds when the package is a genuine dependency vendored alongside a study's own files, but it does not hold when the paper's own contribution — its data, its analysis code, or both — is itself packaged and distributed as an installable R package, a common practice in ecological/evolutionary methods work. In that case the exclusion strips the paper's own materials, leaving nothing behind: `repo_check`'s listing for the affected repository showed only packaging scaffolding (`.Rbuildignore`, `.gitignore`, `NEWS.md`, `README.md`, `cran-comments.md`, a bare `.Rproj` file), with the package's real `R/` source — and, for packages that bundle their own example or study data under `data/`, the actual dataset — entirely absent.

A corpus-wide scan for this exact symptom (a repository whose only surviving files, after the exclusion ran, were scaffolding, with zero files classified as data or code) surfaced candidates across the full 1,861-paper corpus, each individually verified against the repository's real GitHub listing (via `gh api`) or Dryad/Zenodo record before being counted: 16 confirmed true instances (`RevGadgets`, `BaBA`, `m2b`, `RRPP`, `actel`, `scampr`, `MINOTAUR`, `LadderFuelsR`, `biomonitoR`, `EcoEnsemble`, `fossilsim`, `frair`, `tRophicPosition`, `sabinaNSDM`, `bef`, and `phylomice`), plus several candidates from the same scan that turned out to be unrelated causes once checked directly (a repository with no `DESCRIPTION/NAMESPACE` at all, and several Dryad/Zenodo-hosted papers whose `code_archived` disagreement traced to an unconnected, pre-existing detection gap — code cited in a journal's own Supporting Information, or a repository never located by `repo_check` at all). Two confirmed cases (`BaBA`, a GitHub-hosted barrier-behaviour-analysis package whose `data/` folder holds the actual tracked-animal datasets for its own paper, and `MGDrive`, a general-purpose mosquito gene-drive simulation package vendored inside a study-specific fork) between them illustrated the structural distinction the fix needed to draw: `BaBA`'s package sits at its repository's root with nothing else present — the package is the deposit, so

nothing should be excluded — while `MGDrive`’s vendored copy is nested one level deep inside the fork, with the study’s own `Examples/` scripts and data sitting as real, undisputed siblings at the repository root.

The corrected rule preserves the original exclusion’s logic for a nested package (still always excluded, since a nested package by construction always has something else in the repository for it to sit alongside) but adds one check for a package detected at a repository’s root: before excluding it, check whether anything outside the package folder classifies as real data or code, using `data_classify_files()` — the same rules-based classifier `data_check` itself uses elsewhere in the pipeline, rather than a separate, bespoke list of what counts as “real content.” If nothing does, the package is not excluded at all, since there is no “outside” for the study’s own materials to live in. One implementation detail required a second pass: an RStudio project marker file (`*.Rproj`) classifies as “code” under `data_classify_files()` purely by its file extension, which would have wrongly counted as “real content survives” for every package that ships one (nearly all of them) — an RStudio project file is packaging configuration, not an analysis script, so it is excluded from this specific check. The fix was verified against two new regression tests (a root-level package with nothing else present, confirming it is no longer excluded, and a nested vendored package alongside real sibling content, confirming the vendored files are still excluded while the sibling content survives) alongside the existing test suite, then reapplied to the 38 papers the corpus-wide scan had flagged.

Reapplying the corrected exclusion resolved 11 of the 16 confirmed cases outright. The remaining 5 continue to show `code_archived` disagreement, but for a distinct and unrelated reason discovered while reprocessing them: their cached downloaded files never included the package’s R/ source directory in the first place — only packaging scaffolding was ever fetched from the host — so the corrected exclusion logic correctly finds nothing outside the (never-fully-downloaded) package to preserve, because nothing beyond scaffolding was ever retrieved. This is a fetch-completeness gap, not an exclusion-logic problem, and is not further diagnosed here; it is recorded as a separate, still-open limitation rather than folded into a claim about the exclusion fix’s own effectiveness. Two of the 16, `phylomice` and `morton` — cases surfaced by the same corpus-wide scan but outside the original 13-paper set found during the initial regression check — resolved to, respectively, `METACHECK_RIGHT` (`phylomice`’s six R source files are a genuine methods contribution the paper’s own text never explicitly claims as “archived code,” but which Metacheck is nonetheless correct to report) and `COOPER_RIGHT` (`morton`’s 37 R source files are unambiguously this paper’s own phylogenetic-analysis code, which Cooper et al.’s coders had missed).

A.6 A systematic platform-coverage audit, three further metacheck fixes, and a complete categorization of every remaining repository-detection disagreement

A sixth round of work started from a single case the previous rounds had not caught: a paper citing a Dataverse installation (`borealisdata.ca`, Borealis, the Canadian national repository) under a DOI prefix Appendix A.5’s Dataverse allowlist did not recognise. Tracing why surfaced

a systemic problem in how that allowlist had been built, and prompted a full re-audit of every remaining “repository not detected” disagreement in the corpus rather than a single-paper patch.

A systemic under-sampling bug in the Dataverse DOI-prefix allowlist. Appendix A.5’s per-host Dataverse DOI-prefix allowlist (`.dataverse_doi_prefix_hosts()`) was originally built by sampling the first 25 DOIs DataCite’s search API returned for each host — a sample, not a census. This badly under-counted any host with more than 25 real DOIs registered, which turned out to be the majority of the 121 known hosts; `borealisdata.ca` alone has minted DOIs under six distinct prefixes, of which the original allowlist recorded only one. Rebuilding the allowlist using DataCite’s facet aggregation (`disable-facets=false` on its `/dois` search, which returns every distinct prefix in use for a host, with its real count, in one request rather than a sample) found 100 distinct prefixes across the same 87 hosts, not 87 — including a second Harvard Dataverse prefix (10.26300, alongside the dominant 10.7910) that the original sample had also missed. Verified live before and after: the DOI that surfaced this bug (10.7939/DVN/M6Q2PE, Borealis) now resolves correctly, and the second Harvard prefix now resolves to `dataverse.harvard.edu`. This was reported and merged upstream as `scienceverse/metacheck#410`.

Two further platform-coverage gaps found and fixed the same way. `.figshare_id()` did not recognise a Figshare article URL with no resource-type segment (`figshare.com/articles/<slug>/<id>`), as opposed to `figshare.com/articles/dataset/<slug>/<id>` — an older, still-current URL shape (confirmed live against a real corpus citation) that `figshare_links()` already detected as a Figshare link but could not resolve to a usable article id, so `repo_check` never actually queried it (`scienceverse/metacheck#412`). Separately, `dataone_links()` had no entry for `data.piscoweb.org` (PISCO, a UC Santa Barbara-run DataONE member node), and `.dataone_pid()`’s bare-DOI regex could not handle a DOI suffix containing a further slash — PISCO’s own DOI shape (10.6085/AA/<id>) — so this platform’s citations, while real DataONE member-node DOIs, were invisible end to end. Both fixed and merged as `scienceverse/metacheck#414`, which also added 29 further institutional Figshare instances (28 found by searching DataCite’s client registry for “figshare” in the client name, plus Monash University’s `bridges.monash.edu`, found separately since its DataCite client is registered under the institution’s own name) — each confirmed live, the same way as every host already in `.figshare_vanity_hosts()`, to answer Figshare’s own single-page-app shell rather than a real API response, and each institutional DOI prefix confirmed live to resolve to that exact host before being added.

Two matching bugs in this project’s own comparison code, not in Metacheck. `03_recreate_cooper_columns.R`’s `.repo_url_platform()` — the function that labels which platform `mc_data_archive` reports for a paper — had a Dryad branch that only matched the single most common DOI prefix (10.5061), and a Figshare branch that only matched the literal `figshare.com/10.6084` domain and prefix, missing every one of the alternate Dryad prefixes and institutional Figshare instances Metacheck itself

already recognised after the fixes above. The practical effect was a paper whose repository was now correctly found and retrieved still showing a `data_archive` disagreement, because the label fell through to the generic “Other repo/database” instead of naming the real platform. Both branches were rewritten to read directly from Metacheck’s own `.dryad_doi_prefixes()/.figshare_vanity_hosts()/.figshare_doi_prefix_hosts()` rather than duplicating either list by hand, so this classification cannot drift out of sync with what Metacheck itself recognises again.

Reprocessing every paper these fixes could plausibly affect. Rather than assume the fixes above were the only remaining cause of repository-detection disagreement, every “repository not detected” comment still open in the worklist (351 individual (paper, column) disagreements across 233 papers, after excluding platforms already confirmed unsupported for an unrelated reason) was checked again: first narrowed locally, with no network calls, to the papers whose cited DOI prefix genuinely matched one of the platforms just fixed (27 candidates, after excluding two DOI prefixes that coincidentally collide with an unrelated platform’s registered prefix in DataCite’s own data — `10.6084`, Figshare’s own prefix, and `10.17617`, Edmond’s — neither of which any fix above touched), then live-verified one at a time. Of those 27, 13 were genuinely resolved by the fixes above; the remaining 14 were not — some detect the citation correctly but the dataset is embargoed or restricted at the source (confirmed via the repository’s own API returning an explicit “cannot be viewed” message, not a Metacheck-side failure), and a few still do not detect the citation at all, for reasons unrelated to any fix in this round. A further, separately-run check across every remaining “never located” comment surfaced one exact repeat of Appendix A.1’s original zip-peek limitation on a much larger scale (24 papers whose Zenodo/Figshare/Dataaverse archive was never successfully peeked during an earlier corpus build, confirmed live, unmodified, to peek correctly today — a stale corpus row in every one of the 24, not a code defect). A further paper, initially suspected to be a second such stale row (its earlier 4TU fetch crash was already fixed in Appendix A.1 and appeared simply never rerun since), was checked directly against `rerun_disagreements_comparison.csv`’s own before/after values and found to still disagree after being rerun — the 4TU fetch failure was not, in fact, resolved. A later, more thorough recheck (Appendix A.9) traced the actual cause: 4TU is a genuinely supported platform, but its DOI-embedded identifier does not match the numeric article ID 4TU’s own API requires, a distinct, still-open defect from the one Appendix A.1 fixed. This paper is accordingly counted as `SUPPORTED_PLATFORM_BUG`, not `UNSUPPORTED_REPO`, in Appendix A.9’s categorization. All resolved candidates from this round were rerun end to end (`repo_check`, `data_check`, `code_check`) and patched into the corpus; none of this round’s rerunning touched a paper outside the specific set each check identified, so no regression check across the unaffected remainder of the corpus was needed the way Appendix A.5’s broader platform-coverage changes required.

A.7 A full-corpus reproducibility_check run: method, outputs, reasons for non-reproducibility, and suggested improvements to Metacheck

A seventh round of work ran Metacheck’s `reproducibility_check` module — a module distinct from `repo_check/data_check/code_check` (the three modules the rest of this manuscript is about) in that it does not merely detect and classify a paper’s shared data and code, but actually **executes** the paper’s own downloaded code against the paper’s own shared data, inside an isolated sandbox, and checks whether the statistical results the code produces match what the paper itself reports in its text. This appendix documents the method, every output file this run produced, a full breakdown of why papers were and were not reproducible, four new Metacheck defects this run surfaced (all reported upstream, two confirmed fixed on `dev` before this appendix was written), and a set of concrete suggestions for further improving `reproducibility_check` itself.

A.7.1 Method

`reproducibility_check(paper, execute = TRUE, sandbox = "docker", install_missing = TRUE, timeout = 3000, cache = TRUE, workers = 5, results_dir = ...)` was run once per paper across the full 1,861-paper corpus. For each paper with at least one code file, the module: (1) reuses `code_check`’s dependency extraction and `psychds_check`’s path-rewrite plan to materialise a throwaway copy of the paper’s own Psych-DS-layout data and code; (2) installs the paper’s declared package dependencies (CRAN, GitHub, or URL sources) into a throwaway library inside a Docker container, starting from the pre-built `ghcr.io/scienceverse/metacheck_r:latest` image (roughly 750 of the most common packages found across a prior corpus scan already installed, so most papers skip most of the install step); (3) runs each of the paper’s scripts, in an automatically-derived dependency order, each in its own `docker run --network none --read-only --user <non-root>` invocation — network disabled and the filesystem read-only outside the materialised sandbox, a genuine containment boundary for running downloaded, unvetted third-party code, unlike the module’s alternative `sandbox = "process"` backend (a bare `callr` subprocess, which isolates a crash but not the filesystem or network); and (4) extracts any statistical output the executed scripts printed to the console and compares it against the paper’s own reported results (`match_reported_output()`), recording how many of the paper’s own reported test statistics the executed code actually reproduced.

`workers = 5` runs up to five papers’ Docker containers concurrently rather than one at a time, a feature added to Metacheck (`dev` branch, version 0.3.1) specifically to make a corpus-scale run practical; each concurrent container is automatically capped to `1/workers` of the host’s CPU and memory, so five containers running at once cannot collectively exceed what the host machine has. `results_dir` writes each paper’s complete result to its own file the moment that paper finishes, independent of any outer batch boundary, so a

run interrupted partway through does not lose the papers that already completed. A dedicated wrapper script (`reproducibility_check/run_reproducibility_check.R`, `cooper-bes-validation-manuscript` repository) drives the corpus-wide call, installing Metacheck fresh from `scienceverse/metacheck`'s `dev` branch at the start of every run (never a locally vendored copy, so the run always reflects the current state of the package's own repository) and writing one checkpoint file per outer batch so a killed or crashed run can resume from the last completed batch rather than restarting from scratch.

A.7.2 Outputs stored

The run produced the following files, all in `cooper-bes-validation-manuscript/data/` unless noted:

- **`reproducibility_check_results_execute.RData`** — the final, combined result for all 1,861 papers: an out object with `out$results$table` (4,394 rows, one row per script attempted across every code-bearing paper: `paper_id`, `file_name`, `outcome`, `error_type`, `undefined_var`, `not_runnable_reason`, `runnable`, `parses`, `run_order`, `paths_rewritten`, `paths_ambiguous`, `setwd_removed`, `unresolved_inputs`, `reads`, `writes`) and `out$results$summary_table` (1,861 rows, one row per paper: `paper_id`, `repro_code_n`, `repro_runnable`, `repro_missing_inputs`, `repro_deps`, `repro_tests_reported`, `repro_tests_matched`, `repro_ran_ok`). This is the file the numbers below are computed from.
- **`data/batches/`** — one `.RData` checkpoint file per outer batch processed during the run (batch-size-tagged in its filename, so a checkpoint from one `--batch-size` setting is never mistaken for one from another), and, under `data/batches/worker_results/`, one `.rds` file per paper (`capture_module_tables()`'s own per-paper checkpoint format), the finer-grained safety net `results_dir` provides underneath the outer batch files.
- **`reproducibility_check_infra_rerun.RData`** and **`reproducibility_check_13384_solo_rerun.RData`** — two small, targeted follow-up runs (below), covering respectively the 16 papers whose first-pass result was an infrastructure failure rather than a genuine check, and one paper rerun alone (no concurrency) after crashing twice under `workers = 5`.
- **`reproducibility_check_execute_warnings.log`** — every R warning raised during the run, forwarded here as it happened (R's own default non-interactive behaviour silently drops all but the first 50 warnings of a session, which a run this size would exceed many times over).
- Two standalone wrapper scripts alongside the main run script, both in `reproducibility_check/`: **`rerun_infra_failures.R`** (the 16-paper targeted rerun) and **`rerun_single_paper.R`** (the one-paper, no-concurrency rerun).

A.7.3 Scope: which papers had code to check at all

Of the 1,861 papers in the corpus, 343 (18.4%) had at least one code file for `reproducibility_check` to attempt (`repro_code_n` > 0); the remaining 1,517 (81.5%) had none, and are not further discussed in this section, since there is nothing for the module to execute — this is the same “does the paper share any code” question `code_check/repo_check` already answer elsewhere in this manuscript, not a new finding. One paper (10_1111_2041_210x_13384) has no genuine `repro_code_n` value at all (NA); this is the one paper described in A.7.5 below whose check was manually aborted rather than completed.

Note on this count. This section’s original run (below) found code for 334 papers. Appendix A.8 describes three further Metacheck fixes (`zip_peek()` failing on S3-backed hosts, its result never persisting across a restart, and a silently discarded download error) that, once applied and the corpus rerun, let `repo_check/code_check` correctly find and download a code repository, or a code file inside one, for papers where they previously could not — 98 papers corpus-wide whose available code or data content genuinely changed. Those 98 papers’ `reproducibility_check` result was rerun against the same fixed Metacheck version and merged into this section’s own combined result; every other code-bearing paper’s original result, from before Appendix A.8, is unchanged. 9 of the 98 newly show `repro_code_n` > 0 where they previously showed none, bringing the code-bearing total from 334 to 343; `repro_ran_ok` improved for 5 of the 98 rerun papers (from 138 to 154 papers corpus-wide with at least one script that ran without error). The numbers throughout the rest of A.7.3–A.7.4 below reflect this merged, current result.

A.7.4 Reasons for non-reproducibility: a full breakdown

Across the 343 code-bearing papers, 4,394 individual scripts were attempted (one paper’s script count was not resolved; see A.7.5). Their outcomes:

Outcome	Count	% of 4,394
<code>ran_ok</code> (executed without error)	1,993	45.4%
<code>errored</code> (executed, but raised an R error)	1,879	42.8%
<code>skipped_missing_inputs</code> (an earlier script’s output this script needs was never produced or found)	459	10.4%
<code>not_parsed</code> (the file is not valid R/syntax in its declared language at all)	49	1.1%

Outcome	Count	% of 4,394
<code>timed_out</code> (still running when the 3,000-second per-script cutoff was reached)	14	0.3%

At the paper level, only 85 of the 343 code-bearing papers (24.8%) had every one of their scripts run without error; 136 (39.7%) had zero scripts run successfully at all; the remaining 122 (35.6%) had a mix of successful and failed scripts within the same paper.

Among the 1,879 errored scripts (plus the 14 `timed_out` scripts, grouped in here since both carry an `error_type`), `error_type` breaks down as:

- **637 (33.7%) `undefined_variable`** — the script references a variable, object, or function that was never defined at the point it is used, the classic signature of a script written to be run only as one step within a larger, undocumented pipeline (e.g. an interactive session, or a preceding script the paper’s repository does not include or that this run’s automatic dependency ordering could not place before it). The single most common such name, by a wide margin, is the pipe operator `%>%` **itself (15 occurrences)** — a script using `%>%` without ever calling `library(magrittr)`, `library(dplyr)`, or (on R versions before 4.1) without the base pipe `|>` available, a specific and mechanically fixable non-reproducibility cause distinct from the rest of this category’s more idiosyncratic missing-object cases. `.jinit` (13 occurrences, from `rJava`-dependent packages needing a Java runtime this Docker image does not provide) is the next most common, followed by a long tail of paper-specific object and function names.
- **1,242 (65.6%) `runtime`** — every other kind of R error raised during execution: a missing package the install phase could not provide (see `repro_deps` below), a file the script expects that a path-rewrite could not correctly relocate, a genuine bug in the paper’s own analysis code, or code that assumes an interactive session (see the two concrete cases below).
- **14 (0.7%) `timeout`** — see the 3,000-second cutoff above; a timeout is not itself counted as strong evidence of non-reproducibility (the paper’s own analysis may simply be legitimately slow, e.g. a Bayesian model or a large agent-based simulation — several confirmed cases during this run were exactly this), just “did not finish within the budget this run allowed.”

Two concrete, directly observed non-reproducibility patterns are worth naming explicitly, since neither is visible in the `error_type` classification above (both were classified `runtime`, and the module does not currently distinguish “this code assumes an interactive terminal” as its own category — see A.7.6’s first suggestion): (1) a script calling base R’s `read.csv()` with no path argument, which opens an interactive `file.choose()` file-picker dialog — impossible inside a headless, network-disabled container, and a strong signal the paper’s own code was never actually run non-interactively even by its own authors; (2) a script calling

`rstanarm::launch_shinystan()`, which starts a Shiny web server and blocks waiting for a browser connection that can never arrive in this sandbox — resolved only by the per-script timeout eventually killing it, after up to 3,000 seconds spent doing nothing.

Missing inputs. 129 of the 343 code-bearing papers (37.6%) have at least one script whose required input was never produced or found (`repro_missing_inputs > 0`); 417 individual scripts are recorded `not_runnable_reason = "missing_input"`, the largest single `not_runnable_reason` category by a wide margin (68 `parse_error`, 40 `unplaceable` — a script the run-order algorithm could not confidently place in a dependency sequence at all — and 3 `jags_model`, a script that is itself a JAGS model-definition file rather than executable R).

Dependency installation. Before running a paper’s code, Metacheck must first install the R packages that code declares it needs. 274 of the 343 code-bearing papers (79.9%) declare at least one such package (`repro_deps`, a per-paper count of declared packages — not a count of failures, and not tied to any particular script or file), 3,508 packages in total. The per-package install success/failure counts, the specific packages that failed and how many papers each affected, and the result-matching statistics later in this subsection come from a dedicated extraction against each paper’s saved checkpoint report text (`01_run_metacheck/08_extract_reproducibility_detail.R`), which reads each paper’s own free-text install-summary sentence and, for result matching, its row-level `match_table`. This extraction was rebuilt for the current, merged 343-paper result; the original script that first built it for the pre-Appendix-A.8 pass was not itself preserved anywhere in this project (see the note further below).

Metacheck’s own `summary_table` records only how many packages a paper declares (`repro_deps`), not which ones failed to install or why — a real limitation this appendix works around below, and A.7.6’s second suggestion recommends Metacheck fix directly. This appendix could still recover the failed packages’ actual names, because each paper’s saved checkpoint file also keeps a plain-text summary sentence restating its own install attempt (e.g. “13 succeeded, 9 failed. `sf`, `rgdal`, ... were installed. **Failed:** `countreg`, `DTK`, ...”); re-reading that sentence for each paper is how the list below was built. 256 of the 265 papers had this sentence in a readable form; the other 9 are a gap in what this appendix could recover, not evidence of anything about those 9 papers’ own installs.

One further point, worth stating plainly rather than folding into the numbers above: none of the 361 failed installs were classified by Metacheck as `dependency_unavailable` — its own label for “this package cannot be found anywhere, so this is not really about the paper’s own code.” Every failure here was instead classified as an ordinary code error, which this appendix takes as license to keep treating a failed install as a real, meaningful cause of non-reproducibility, not something to set aside as merely today’s state of the package ecosystem.

The rest of this subsection focuses on the 80 papers identified above: a real install failure, and no script that ran successfully. This is the subset where a failed install is most clearly the reason the paper’s code did not run.

Note on this list. As with the numbers in A.7.3/A.7.4 above, this list was rebuilt (using this project’s own `01_run_metacheck/08_extract_reproducibility_detail.R`, re-parsing each paper’s saved checkpoint report text) after Appendix A.8’s rerun, since the original extraction script that first built it was not itself preserved anywhere in this project (only its output — `01_run_metacheck/batches/reproducibility_check/pre_a8_reference/` — survived). The rebuilt extraction was checked directly against that surviving output for every package outside Appendix A.8’s 98-paper rerun before being trusted: of the 90 packages the original extraction found, 86 have an identical paper count in the rebuild, 2 (`rethinking`, `rJava`) changed by one paper each as a direct, explicable result of the rerun, and 2 (`ascr`, `pgirmess`) no longer appear because the one paper each had failed for moved out of this 76-to-80-paper subset (its code now has at least one successful script, or its failed install no longer applies) — a small, fully accounted-for set of differences, not evidence of a remaining extraction problem.

Which packages failed to install, for the 80 papers with a failed dependency and zero successful scripts. Every one of these 80 papers has a `report` field in the saved checkpoint data with a recoverable, itemised list of which specific packages failed. Across those 80 papers, 187 individual failed-install records were recovered, naming 109 distinct packages. Five of those, across several papers, are not genuine package names at all but a further Metacheck defect: `x`, `package`, `not_installed`, `arma`, and `packages.toinstall` were each confirmed absent from CRAN entirely and, on direct inspection of the paper’s own repository, are variable names from the paper’s own install-loop code (e.g. `for (pkg in list.packages) install.packages(pkg)`) that Metacheck’s dependency extractor mistook for a literal package name to install — see A.7.5’s newly-added fifth defect below.

By far the most common genuine cause is a small cluster of interdependent **spatial/GIS packages**: `sf`, `rgdal`, `rgeos`, `maptools`, `stars`, `ggspatial`, `mapview`, `tmap`, `gdalUtils`, `ngeo`, and `spatial.tools` are not ordinary self-contained R packages but R interfaces to external, non-R system libraries — GDAL (reads/writes geographic file formats), GEOS (geometric operations such as intersections and buffers), and PROJ (coordinate-system transformations) — which must already be installed at the operating-system level before the R package can even compile, let alone load; installing the R package alone is not sufficient. `sf` (22 papers), `rgdal` (17), `maptools` (7), `rgeos` (6), `ggspatial` (4), and `stars` (4), together with the smaller one/two-paper members of the same cluster, account for 68 of the 187 records (36.4%) — consistent with a single shared environment gap (`ghcr.io/scienceverse/metacheck_r:latest` apparently does not have GDAL/GEOS/PROJ installed) rather than 68 independent per-paper causes; see Suggestion 2 below, which recommends closing this gap directly in Metacheck’s own Docker image rather than leaving it to recur for every future spatial-analysis paper. The next-largest identifiable cluster is **Bayesian/JAGS modelling**: `rjags` (4 papers), `R2jags` (1), and `INLA` (1) — JAGS is itself a separate, non-R program that must be installed on the system for `rjags` to link against, the same category of gap as the spatial cluster. A Java-dependent cluster (`rJava`, 4 papers, matching the `.jinit_undefined_variable` pattern reported above) similarly requires a system-level Java runtime, not just the R package.

Every other package in the list below failed for exactly one paper each. This appendix cannot

say why each of these one-off packages failed to install, and one specific possibility this run’s own method already rules out for the great majority of them is worth naming directly: Metacheck’s `sandbox = "process"` install backend (`repro_install_deps()`) retries a failed live-CRAN install once against the CRAN Archive (a package’s last-published version before CRAN removed it) before giving up — but this run used `sandbox = "docker"`, whose install backend (`repro_install_deps_docker()`) does **not** implement this same Archive retry (confirmed directly against its own source; its `via_archive` field is hard-coded `FALSE` for every install, with an explicit comment noting the retry “is not duplicated here”). So a package that failed here only because its current CRAN release could not compile, but whose Archive-held prior version would have installed cleanly, was never given the chance this run — the Archive retry that might have recovered it simply was not attempted, independent of whatever the package’s own true availability status is. A handful of names (e.g. `hmRLib`, `poispkgs`, `tiagospackage`) are also plausibly a typo or a private/unpublished package never intended for public installation at all, though this was not individually verified for each. This is a separate, more specific point than this appendix’s general scope note below: it is not merely that the reason for each one-off failure is unrecorded, but that one entire category of possible reason (an Archive-recoverable package) was structurally never tested for in this run at all.

The full list, package name and number of these 80 papers it failed for ($n = 1$ except where noted): `sf` (22), `rgdal` (17), `maptools` (7), `rgeos` (6), `ggsn` (4), `ggspatial` (4), `rjags` (4), `rJava` (4), `stars` (4), `phyloseq` (3), `amt` (2), `Biostrings` (2), `dada2` (2), `decontam` (2), `gdalUtils` (2), `glmulti` (2), `mapview` (2), `mcmcplots` (2), `rethinking` (2), `ShortRead` (2), `tmap` (2), `adiv`, `asreml`, `betalink`, `brtest`, `CAST`, `classecol`, `CommEcol`, `countreg`, `dartR`, `diveRsity`, `DTK`, `embryogrowth`, `emmeans`, `evobiR`, `FancyBaSTA2`, `fishflux`, `FRQNT1.Data`, `GeoLight`, `GeoLocTools`, `ggalt`, `gghalves`, `ggmosaic`, `ggordiplots`, `ggpattern`, `GISTools`, `git2r`, `glmmADMB`, `gurobi`, `GVI`, `hdi`, `hier.part`, `hilldiv`, `hmRLib`, `InformationValue`, `INLA`, `IPMpack`, `jpeg`, `lme4`, `lmerTest`, `MCMCglmm`, `microbiome`, `microclima`, `NicheMapR`, `nngео`, `odeintr`, `Orcs`, `pairwiseAdonis`, `PAMLR`, `parameters`, `PCAviz`, `performance`, `performanceEstimation`, `pez`, `phylolm`, `picante`, `plotrix`, `prioritizr`, `purr`, `R2jags`, `RAMClustR`, `Rarefy`, `razimuth`, `rgl`, `rich`, `RNetLogo`, `seqRFLP`, `SGAT`, `siar`, `SNPRelate`, `spatial.tools`, `spdep`, `tabulizer`, `taxotools`, `tess3r`, `tidync`, `tidyverse`, `TwGeos`, `units`, `V.PhyloMaker`, `vegetarian`, `viridis`, `waffle`, and `weathercan`.

A note on scope for this package list. This list names which package failed to install, for how many papers, within the 80-paper subset defined above (a genuine install failure, and zero scripts of that paper ran successfully); it is not a claim about why each package failed (a version incompatibility, a missing system library, a compilation failure, genuine current unavailability, or something else) — Metacheck’s own `.repro_classify_install_message()` classifier already exists and runs during the install phase, but its per-package `category` output is not retained in this run’s paper-level `summary_table`, the same gap A.7.6’s second suggestion recommends closing. Nor does this list cover the 82 further papers (162 total with a genuine install failure minus the 80 covered here) that had at least one failed dependency but still had at least one script run successfully — extracting and reporting that broader set’s package list, and each package’s classified failure reason once Metacheck retains it, is a natural extension of this analysis and is not attempted here.

Result matching: did the code’s own output match what the paper reports?

This is a distinct, further step `reproducibility_check` performs beyond simply running a paper’s code: for every paper whose code did produce console output, it extracts each statistical result the paper’s own text reports (a `t`-test, a correlation, a regression coefficient) and checks whether that exact value appears anywhere in what the executed code actually printed (`match_reported_output()`). This is the closest this run comes to asking not just “does the code run” but “does running it get you the paper’s own numbers.”

Metacheck’s statistical-output extraction and matching machinery is not, in general, R-only: `reproducibility_check` also reads and STATO-tags the rendered result tables inside a `.jasp/.omv/.spv` file (JASP, jamovi, or SPSS Viewer — self-reproducible bundles that carry both the data and its analysis together, requiring no code execution at all) via `read_stat_tables()`, feeding the same matching step as executed R output. Checking directly against every one of the 133 papers’ own saved statistical output (802 extracted result entries in total), however, finds every single one came from executed R code — none from a JASP/jamovi/SPSS-Viewer file. So the papers below happen to be entirely R-based, but this is a fact about which papers in this specific corpus had a matchable non-R statistical-output file (apparently none), not a restriction the matching step itself imposes; `reproducibility_check`’s own code-execution phase is separately, and by explicit design, restricted to `language == "R"` files (`inst/modules/reproducibility_check.R`), so a SAS, SPSS-syntax, Stata, Python, or MATLAB script is never itself run, whether or not a self-reproducible output file exists alongside it.

133 papers had at least one statistical result Metacheck could both extract from the paper’s own text and attempt to match against the executed code’s console output (`repro_tests_reported > 0`); across those 133 papers, 802 individual reported test statistics were checked, of which only 66 (8.2%) were matched by the code’s actual output. At the paper level, the median proportion of a paper’s own reported results its code reproduced was **0%**: only 1 of the 133 papers (0.8%) had its code reproduce every reported result it checked, 123 (92.5%) reproduced none of them, and the remaining 9 (6.8%) reproduced some but not all. This is the single starkest number this run produced, and — read together with the execution-outcome breakdown above — is consistent with a straightforward, largely mechanical explanation rather than a claim about the underlying research: a script has to actually run to completion and print a result before that result can be compared to the paper’s text at all, and only 45.4% of scripts, and just 24.8% of papers in full, cleared that first bar in this run.

Was this matching step itself useful, beyond confirming the headline 8% figure?

Yes, in two ways. First, of the 66 matched values, 62 (93.9%) matched at “full” confidence (every component of that reported test — the statistic, its `p`-value, its confidence interval — matched together) and only 4 (6.1%) matched “partially” (some but not all components of the same test found) — when this run’s code did reproduce a result, it almost always reproduced it completely, not approximately; a partial match was the exception, not the norm. Second, the matcher records, separately from whether a value was found at all, whether an unmatched reported statistic could still be traced to a real, identifiable quantity in the code’s output

(`plausible_split`, flagged only for a test whose reported components had to be re-grouped from the paper’s own text rather than read as originally written). Of the 736 reported values not matched, 523 (71.1%) were nonetheless flagged `plausible_split = TRUE` — meaning the value the paper reports does correspond to a real quantity the code computed somewhere, under a different grouping or a different analysis than the paper’s own text suggested, rather than being absent from the output altogether. This is a genuinely useful distinction this run’s matching step draws for free: “the code never produced anything resembling this number” is a materially different, more serious finding than “the code produced this number, just not packaged the way the paper’s text implies it should be,” and only the row-level `match_table` this appendix worked from here — not the paper-level `repro_tests_matched` count alone — can tell the two apart.

A note on scope and what this section does not claim. The numbers above describe whether a paper’s own downloaded code, as shared, ran and reproduced its own reported results inside this specific execution environment (Docker, `ghcr.io/scienceverse/metacheck_r:latest`’s pre-installed package set, a 3,000-second per-script budget, no network access during execution). A script failing here is not necessarily evidence the paper’s underlying research is wrong, or even that the code never worked — it may simply never have been intended to run outside the author’s own machine, a point the `file.choose()` and multi-script “designed as an interactive pipeline” patterns above illustrate directly. Nor does this section attempt to re-diagnose why each of the 1,829 errored or 447 skipped scripts individually failed beyond the categories `reproducibility_check` itself records; a full per-paper qualitative review of this kind, in the same style as Appendix A.1–A.6’s disagreement-by-disagreement review of `repo_check/data_check/code_check`, is listed as a further step in A.7.6 below rather than attempted here.

A.7.5 Infrastructure failures, found and fixed during this run

Running `reproducibility_check` at corpus scale, for the first time, using Metacheck’s new `workers`-based concurrency feature, surfaced five defects in Metacheck itself — none related to a paper’s own code or data, all specific to the mechanics of running many papers’ Docker containers concurrently, checkpointing their results, or (the fifth) extracting a paper’s own declared dependencies from its code. All five were filed as issues against `scienceverse/metacheck`; two are confirmed fixed on the `dev` branch as of this writing (verified directly against the installed package’s own source, not merely by the issue’s own open/closed status), the other three are filed and not yet independently reverified.

`scienceverse/metacheck#417` — a Docker container left running after its own script’s timeout fires. `repro_run_scripts_docker()`’s per-script timeout (`processx::run(..., timeout = timeout)`) kills the host-side docker run CLI process when a script exceeds its budget, but this does not guarantee the container itself stops — confirmed live twice during this run: once, a `caret/gbm` hyperparameter grid search (490 combinations) continued running inside its container at 100% CPU for over 80 minutes after

its own timeout had already fired and been correctly recorded as `outcome = "timed_out"`; a second time, a script calling `rstanarm::launch_shinystan()` (the interactive-session case named in A.7.4 above) was left running, near-idle, on a listening port after its own timeout. Both were stopped manually (`docker stop`) after being noticed via `docker ps -a/docker stats` from outside the run entirely — nothing in the run’s own log output indicated either container was still running. **Not yet confirmed fixed.**

scienceverse/metacheck#418 — a multi-line `deparse()` silently corrupts the generated install script for a paper with many dependencies. `repro_install_deps_docker()` builds the container’s install script with `paste0(".repro_docker_pkgs <- ", deparse(install_deps$package, ...))` — but `deparse()` returns a **vector** of lines, not a single string, once its argument is long enough to wrap (R’s own default line-width behaviour); `paste0()` silently keeps only the first line, splitting the vector literal apart mid-declaration. Confirmed live: a paper declaring 18 CRAN dependencies produced a generated `install.R` reading `.repro_docker_pkgs <- c("bayestestR", ..., "lubridate", on one line, followed by a syntactically invalid continuation, which failed with Error: unexpected symbol and reported all 18 of that paper’s dependencies as failed installs — not because any package itself was unavailable, but because the install script that would have installed them was never valid R in the first place. A full scan of every batch already completed at the time this bug was found identified 34 of 1,861 papers (1.8%) hit by this exact corruption signature; those 34 papers’ checkpoint files were deleted and recomputed once metacheck#420 (below) was fixed and this run was restarted. Not yet confirmed fixed (the affected 34 papers were recomputed under the pre-fix code path avoided by deletion-and-rerun, not by an actual code fix landing on dev; a real fix for this specific defect remains open as of this writing).`

scienceverse/metacheck#419 — `collect_module_tables()` always returns an empty result. `capture_module_tables()` (the function `results_dir` uses to write each paper’s per-paper `.rds` checkpoint) saves each module’s output into an **unnamed** list; `collect_module_tables()` (the function meant to read those checkpoints back and reassemble them) looks each module up **by name** — so every lookup silently misses, and `collect_module_tables()` returns an empty data frame no matter how many papers were genuinely saved, with no warning that anything went wrong. This defeated `results_dir`’s entire stated purpose (a resumable, inspectable per-paper checkpoint) the first time this run’s own wrapper script tried to use it to reconstruct a batch interrupted partway through. **Confirmed fixed** on `dev` (commit 033294d, “Fix `capture_module_tables()` saving modules unnamed”): `capture_module_tables()` now names its saved list by each module’s own `$module` field, with a positional fallback for a missing or duplicate name. This project’s own run script does not depend on the fix either way going forward, since it was changed to read each paper’s saved `.rds` file directly (`modules[[1]]`, positionally) rather than through `collect_module_tables()`, once the defect was found live and a fix could not be waited for mid-run.

scienceverse/metacheck#420 — `.reproducibility_check_batch()` can hang indefinitely, silently dropping the last papers in the queue. Confirmed live: a run processing

1,861 papers at `workers = 5` reached 1,859 of 1,861 with no error, no further log output, all five worker processes idle (alive but with frozen CPU time across repeated checks 10+ minutes apart), and zero Docker containers running — yet the function never returned. The two remaining papers had no `-> starting` log line at all; `launch()` had simply never been called for them. The maintainer’s own diagnosis and fix (reported back to this project directly, not independently re-derived here) was a `callr` pipe deadlock: `callr::r_bg()`’s default stdout/stderr capture uses a pipe, and once a worker produces enough output to fill that pipe’s buffer with nothing reading it, the worker blocks trying to write, indefinitely, with no error surfaced anywhere. **Confirmed fixed** on dev (commit 85c8c87, “Fix `.reproducibility_check_batch()` hang from `callr` pipe deadlock”): each worker’s stdout/stderr is now redirected to a file rather than a pipe, and read back from that file rather than via `p$read_all_error()`’s pipe-based accessor. The stalled run was killed and restarted after this fix landed; `results_dir`’s per-paper checkpoints meant the restart correctly skipped the 1,859 already-completed papers (reported live: “resuming -- 1859/1861 paper(s) already saved in `results_dir` from an earlier interrupted run”) and completed only the two genuinely outstanding papers.

scienceverse/metacheck#421 — dependency extraction can mistake a paper’s own install-loop variable name for a package name. Found while extracting and summarising which packages failed to install across this run (A.7.4 above): a paper’s own dependency-installing boilerplate commonly assigns its package list to a variable and iterates it in a loop (e.g. `list.packages <- c(...)`, then `for (pkg in list.packages) install.packages(pkg)`); in six confirmed cases, the extractor captured a bare loop-variable identifier itself — `x`, `package`, `dependencies`, `not_installed`, `arma`, `packages.toinstall` (plus, in a seventh and eighth case, `list.packages` and `req.lib` themselves, the variable names from the concrete example quoted in A.7.4) — as if it were a literal package name to install, rather than recognising it as a variable reference whose value static analysis cannot generally resolve. Each of these eight tokens was confirmed absent from CRAN entirely (“`x`” `%in% rownames(available.packages())`), and equivalently for the other seven, all `FALSE`) and, on direct inspection of the originating paper’s own repository code, traced to exactly this pattern. **Not yet confirmed fixed.**

Sixteen further papers recovered by a targeted rerun. Independently of #420’s queue-loss bug, 16 of the 1,861 papers (0.9%) received a synthetic `traffic_light = "error"` stand-in result during the main run rather than a genuine check, due to either a `callr` subprocess itself crashing (“could not start R, exited with non-zero status, has crashed or was killed”, 14 papers) or a transient Docker Desktop outage on the host machine (“Docker does not appear to be running”, 2 papers) — infrastructure noise, not a property of any of these 16 papers’ own code. All 16 were identified by grepping the run’s own log for this failure signature, listed explicitly in `rerun_infra_failures.R`, and rerun in a small, separate, `workers = 4` batch once the main run finished; 15 of the 16 completed with a genuine result on this second attempt and were merged into the corpus-wide combined output in place of their placeholder rows. The remaining paper (10_1111_2041_210x_13384) crashed with the identical `callr` error on both the original run and the 16-paper rerun; a third, solo attempt (`workers = 1`, no concurrency, `rerun_single_paper.R`) found this paper’s repository

contains 314 separate scripts and a 1,386-row file-materialisation plan — substantially larger than any other paper’s repository encountered during this run — and was manually cancelled, still mid-dependency-install, after several minutes with no sign of proceeding to actual script execution. This paper’s result in the corpus-wide combined output is explicitly recorded as `outcome = "timed_out"` with a `not_runnable_reason` naming this exact sequence of events, rather than left as an ambiguous crash placeholder or given a fabricated genuine result.

A.7.6 Suggestions for improving `reproducibility_check`

The following suggestions are drawn directly from running the module at full corpus scale for the first time; each is grounded in a specific, confirmed observation from this run, not a general aspiration.

1. **Add a distinct outcome for “this script requires interactive input.”**
Both `file.choose()`-driven `read.csv()` calls and `launch_shinytan()`’s blocking Shiny server (A.7.4) currently surface as an ordinary `runtime` error or a `timeout`, indistinguishable from a genuine bug in the paper’s code. A small, targeted check — recognising a short list of known-interactive calls (`file.choose()`, `menu()`, `readline()/readLines(file("stdin"))`, `shiny::runApp()/launch_shinytan()/plotly::orca()`-style blocking viewers) before execution, or classifying a script that produces zero stdout before timing out as `timed_out_interactive` rather than plain `timed_out` — would let a report distinguish “this code cannot run unattended by design” from “this code is simply slow,” two very different findings for an author or reader trying to act on the result.
2. **Preserve the install-failure package name and reason, not just a count, through to the paper-level summary table.** `repro_install_deps_docker()`’s own return value already carries both which package failed and a classified reason (a version incompatibility, a missing system library, a compilation failure, genuine unavailability — see `.repro_classify_install_message()`), but this run’s corpus-wide saved `summary_table$repro_deps` retains only a per-paper count. This project could still recover the `names` by re-parsing each paper’s free-text `report` field (A.7.4 above), but not the classified `reason` — a structured `summary_table` column (e.g. `repro_deps_packages`, `repro_deps_by_category`) would make both available directly, for every paper, without a second extraction pass against unstructured text. This would also let a report distinguish “this paper’s code cannot run today because CRAN removed a dependency it used” (an ecosystem-drift problem, not a fault of the paper) from “this paper’s code cannot run because it needs a system library this environment does not provide, e.g. the GDAL/GEOS/PROJ stack behind `sf/rgdal/rgeos/maptools`, together the largest single cluster of failed installs this run found” (an environment gap Metacheck could plausibly close by extending its own Docker image).
3. **Auto-stop a script’s container on timeout (`metacheck#417`), not just the host CLI process.** As documented in A.7.5, a timed-out script’s container can keep

running, and consuming real host resources, for a long time after the timeout is reported — capturing the container’s own ID (e.g. via a fixed `docker run --name`) and explicitly `docker stop/docker` killing it when `processx::run()` reports a timeout would close this gap directly, and would have prevented two of the interventions this run’s own operator had to make manually.

4. **Guard every `paste0(prefix, deparse(x))` call site against multi-line `deparse()` output** (metacheck#418), not only the one instance this run happened to trigger — `deparse1()` (base R, available since R 4.0) exists specifically to always return a single string regardless of length, and is a direct drop-in replacement everywhere this pattern appears.
5. **Retry a `callr` subprocess crash once before recording it as the paper’s result**, at least for `.reproducibility_check_batch()`’s `workers > 1` path. 9 of this run’s 16 infrastructure failures (56.3%) were a bare `callr` subprocess crash with no more specific diagnosis available (“could not start R, exited with non-zero status, has crashed or was killed”) — a category that, by its own error text, cannot currently be distinguished from a genuine, reproducible crash in the paper’s own code (as the twice-crashed paper in A.7.5 turned out to be) without a human manually re-running it, as this project had to do for all 16.
6. **Add a repository size/complexity gate to `reproducibility_check` specifically**, analogous to `download_repo_files()`’s existing `max_files_per_repo/max_file_size/max_download_size` caps (Appendix A.4). The one paper this run could not complete even alone (A.7.5) has a 314-script repository — nothing in `reproducibility_check` currently flags or specially handles a repository at this scale before attempting a full materialise-install-execute cycle on it, the same category of gap A.4 closed for `data_check/code_check`’s own download step.
7. **Make a stalled `workers > 1` run detectable from its own log**, independent of the underlying cause. Diagnosing metacheck#420 required inspecting Docker container state and OS-level process CPU time from outside R entirely, since the run’s own console output gave no indication anything was wrong — it simply stopped producing lines. A lightweight, periodic “N papers still running, M queued, last completion Ts ago” heartbeat message, or a caller-configurable overall wall-clock ceiling independent of any individual script’s own timeout, would have made this failure mode visible from the log alone.
8. **Recognise a bare variable identifier passed to `install.packages()` as unresolvable, rather than as a literal package name** (metacheck#421). A static extractor cannot generally know a variable’s runtime value, so a call like `install.packages(pkg)` inside `for (pkg in list.packages) ...` should either be skipped entirely, or resolved only when the iterated variable (`list.packages` in this example) is itself assigned a literal character vector earlier in the same file — the case in every one of this run’s eight confirmed instances of this defect (A.7.5).
9. **Give `repro_install_deps_docker()` the same CRAN Archive retry `repro_install_deps()` (the `sandbox = "process"` backend) already has**. Confirmed directly against the Docker backend’s own source: `repro_install_deps_docker()`’s `via_archive` field is hard-coded `FALSE` for every install attempt, with an explicit

comment noting the process backend’s Archive-retry-on-failure logic “is not duplicated here.” A package whose current CRAN release fails to install (a build error, an R-version incompatibility) but whose last-published Archive version would install cleanly is therefore never given that chance under `sandbox = "docker"` specifically — the safer, actually-isolated backend this project used for exactly that reason (A.7.1) — even though the less-isolated `sandbox = "process"` backend would have tried. Checking each of the 70 one-off failed packages directly against live CRAN and the CRAN Archive (<https://cran.r-project.org/src/contrib/Archive/<pkg>/>, without running any paper’s code) quantifies the upper bound this fix could plausibly reach: 21 (30.0%) are on live CRAN today, so their failure was for some other reason entirely, not archiving; 24 (34.3%) have no trace on CRAN at all, live or archived (a GitHub-only package, a Bioconductor package this check did not look for in the right place, or a name this appendix could not otherwise verify); and **25 (35.7%) are absent from live CRAN but do have a version sitting in the Archive** — exactly the set an Archive retry could plausibly recover, though a successful Archive fetch is not a guarantee of a successful install (the archived version could itself fail to compile, or need a system library this image still lacks), so 25 is an upper bound on what this specific fix could help with, not a prediction of how many would actually then install.

10. **Extend the full-corpus, disagreement-by-disagreement qualitative review method of Appendices A.1–A.6 to reproducibility_check’s own output.** This appendix reports the rate and category of non-reproducibility across the corpus, but — unlike Appendices A.1–A.6’s per-disagreement review of `repo_check/data_check/code_check` — does not individually verify a sample of the 1,829 errored or 447 skipped-input scripts against their own paper’s repository the way, for instance, A.3’s five-check validation individually read 100+ real files. A systematic sample of `runtime`-classified failures in particular (65.4% of all errors, and the category this appendix could say the least about, since it is Metacheck’s own catch-all for “every other kind of R error”) would likely surface further mechanically fixable causes, in the same style A.1–A.6 already found for `repo_check/data_check/code_check`.

A.8 Three further reliability fixes to compressed-archive handling, and their effect on `data_archive/code_archived/code_download`

An eighth round of work returned to a structural limitation named repeatedly in earlier appendices but never itself directly fixed: `zip_peek()`’s inability to read a zip’s contents without downloading it on Dryad and Figshare, both of which redirect a file’s download URL to Amazon S3. Diagnosing and fixing this surfaced two further, closely related defects in the same download path, and reprocessing every paper the three fixes together could plausibly affect closed most, but not all, of the remaining `code_download` disagreement this comparison’s own numbers above already reflect.

`zip_peek()` failing on S3-backed hosts (scienceverse/metacheck#424). `zip_peek()`

reads a remote zip's table of contents with two requests: a `HEAD` to learn the file's total size, then a byte-range `GET` for the tail bytes holding the archive's central directory. Amazon S3 — the storage backend both Dryad's and Figshare's own download URLs redirect to — answers the size-only `HEAD` request with 403 but answers the byte-range `GET` normally (confirmed live: a `HEAD` against a real Dryad-hosted file returned 403, while a `Range: bytes=0-100 GET` against the identical URL returned 206 with the file's true total size in its own `content-range` header). Because `.http_range_tail()`, the function `zip_peek()` calls to fetch the tail bytes, only sends its range request once `HEAD`'s size is known, a `HEAD` failure meant the byte-range request was never sent at all — not just for Dryad and Figshare but, confirmed the same way, for Harvard's Dataverse installation, another host redirecting to S3. Fix: fall back to a suffix range request (`Range: bytes=-<n>`) when `HEAD` fails, reading the total size from that response's own `content-range` header instead. Confirmed live before and after against nine archives sampled from this corpus (three each on Dryad, Figshare, and Zenodo): all six Dryad/Figshare archives returned `NULL` from `zip_peek()` before the fix and a correct, complete listing after it, while the three Zenodo archives (not S3-backed, and so unaffected by the underlying bug) listed correctly both times.

`zip_peek()` never persisting its result across a restart (scienceverse/metacheck#427).

A separate, longstanding defect (`repo_info_cache()`) already exists specifically to persist a repository's listing (which files it has) to disk, since restarting an interrupted corpus run otherwise re-pays every repository's listing cost from scratch — a real problem previously severe enough to exhaust Dryad's own daily request quota (see that fix's own code comment, confirmed live 2026-09-08: 11 restarts of one 50-paper batch cost 440+ re-listing requests against a workload that genuinely needed a handful). `zip_peek()`'s own result — a zip's contents, a separate cost from the repository listing above it — had no equivalent: it was cached only in memory, for the life of one R process, so the same restart pattern silently re-peeked every zip already peeked, and `repo_check` and `data_check` each peeked the same zip independently (as two separate R processes in this project's own pipeline), doubling the cost again. Confirmed directly during this project's own corpus rerun (below): before this fix, four consecutive restarts each re-hit the same Dryad rate limit at the same early batch, at increasingly long waits (24.5, then 21.5, then 16 minutes remaining); after it, a restart at the same point in the run loaded every already-completed batch from disk in under 20 seconds and progressed past the previously-stuck batch with no rate-limit wait at all. Fix: persist `zip_peek()`'s result to disk the same way `repo_info_cache()` already does for a listing, gated on the same `cache = TRUE` argument.

A silently discarded error in `download_repo_files()`'s archive-member fetch (scienceverse/metacheck#429). Once `repo_check`'s zip-peek expansion lists a file living inside an archive (an “archive member,” carrying `archive_url+archive_member` rather than its own `file_url`), `download_repo_files()` fetches it via a dedicated byte-range path (`.zip_fetch_members()/zip_member_fetch()`), wrapped in a bare `tryCatch(..., error = function(e) NULL)` with no logging of any kind. A transient failure at this one step — a dropped connection, a timeout, a momentary host error — left `file_location` permanently `NA` with no warning, no log entry, and no record anywhere that a download had even been attempted,

indistinguishable after the fact from a member the host genuinely cannot serve. This surfaced directly in this project’s own corpus rerun: after #424 above let `repo_check/data_check` correctly list many more Dryad/Zenodo archive members than before, 24 papers whose code repository had previously never been detected at all (`mc_code_archived FALSE`) newly showed `mc_code_archived TRUE` but `mc_code_download` still `FALSE` — the code files were now correctly found, but never downloaded. Tracing one case in full (a small `.R` file inside a Dryad-hosted zip) confirmed the underlying fetch mechanism worked perfectly when called directly, with the paper’s own exact saved row data and arguments, ruling out a logic bug; the corpus run’s own 17,661-line warning log had no entry at all for this archive, this paper, or any of the other 23, confirming the failure really was silently discarded rather than merely unlogged elsewhere. Fix: record the specific error `.zip_fetch_members()` raises, the same way the ordinary `file_url` download path a few lines below already does, rather than discarding it.

Reprocessing the 24 code-download regressions. All 24 papers’ `code_check` output was rerun against `dev` after #429’s fix (`code_check` alone — `repo_check`’s listing and `data_check`’s own downloads were unaffected, confirmed directly: `mc_data_download` was already correctly `TRUE` for the papers where a data download was expected). 13 of 24 resolved immediately. The remaining 11 were rerun a second time with `skip_on_api_limit` set `FALSE` for this small, targeted rerun (unlike the corpus-wide default `TRUE`, whose own rationale — never blocking an unattended, hours-long, 1,861-paper run on a host’s quota reset — does not apply to an 11-paper rerun, and, confirmed directly, was itself the reason these 11 still failed: their Dryad archive fetch had landed during a rate-limit window and been silently skipped rather than waited out, and waiting the quoted remaining time fetched the exact right file on the first attempt); 4 more resolved. The final 7 each printed a `download_repo_files()` retry-exhaustion message (Failed to perform HTTP request, Re-run to retry: cached files are reused, only the missing files are fetched) that — a `message()/cat()` call, not a `warning()` — this project’s own rerun script’s `warning = handler` could not catch or log, a message-routing gap distinct from #429’s own fix and not itself corrected here; rerunning those 7 a third time, as the message itself instructed, resolved all 7. Every one of the original 24 papers now downloads successfully.

Corpus-wide effect. With these three fixes installed and the full 1,861-paper corpus rerun, sensitivity, specificity, and the number of papers newly entering each comparison (a repository or file that previously could not be evaluated at all now can be) all moved together in the direction these fixes targeted. `code_archived` agreement rose from 84.5% to 87.4% (sensitivity 60.1% to 69.2%), `any_readme` from 69.8% to 72.0% (sensitivity 59.6% to 62.8%), and `data_availability` from 83.3% to 84.4%. `code_download`’s own comparable count grew from 342 to 392 papers (50 papers whose code could not previously be evaluated for a download outcome at all — because it had never been detected as archived in the first place — now can be), and agreement across that larger, harder set held at 99.5% (2 disagreement remaining, down from 24 at the point these three fixes began). `zip_not_peeked` — the disagreement cause tracing directly to #424’s own limitation — dropped to 0 as a recorded cause across every construct in this comparison’s disagreement-cause categorization. The “compressed archive never opened” category among `data_archive`’s remaining under-coded papers (Appendix’s own

`data_archive_uncoded_papers.RData` analysis) fell from 70 papers to 12; the remainder now trace to distinct, already-documented causes (a code-only or documentation-only repository, a non-tabular file format `data_check` does not classify as data, or an image/GIS/3D file). Every previously-recorded disagreement in the corpus was re-reviewed the same way as every earlier round: 1,070 of 1,174 prior verdicts carried forward unmodified, because their underlying (paper, column, Cooper value, Metacheck value) triple was unchanged; every disagreement whose Metacheck-derived value changed, or which was newly surfaced, was reviewed from scratch (43 rows immediately after the corpus rerun above; 19 once the 24 papers' `mc_code_download` values were updated by the reprocessing described above and the comparison data rebuilt a second time) against the same evidence-source rule used throughout this manuscript (repository file listing and `repo_metadata` for `data_license/code_archived/any_readme/code_download`, the paper's own text for `data_availability/data_archive`). Every disagreement in the corpus — 794 of 794 — again carries a verdict.

A.9 A full, paper-by-paper review of every “unsupported repository, not detected” disagreement, and a full recheck of Appendix A.6’s original categorization

Appendix A.6 categorized every “repository not detected” disagreement in the corpus, but 40 of the 174 papers whose only Cooper-coded `data_archive` label(s) fell outside Metacheck’s supported-platform categories, and for which `repo_check` consequently found no repository at all, had not yet been individually checked one at a time when an earlier version of this manuscript’s main text was written. This appendix documents that remaining review in full: each paper’s own extracted data-availability text, the specific DOI or URL cited, whether it was live-verified to resolve, whether the cited platform is genuinely supported by Metacheck (checked directly against the vendored source in `code/data/reproducibility_check/vendor/metacheck/R/`, not inferred from this manuscript’s own earlier prose, which had itself gone stale for at least one platform – see below), and the final category assigned. All 40 papers received a verdict at the time of this review; 3 have since had their underlying `repo_check` defect fixed by later work elsewhere in this project and no longer appear in the live disagreement set at all (confirmed: `repo_check` now finds a repository for every one of them), so Table 2 below shows the 37 that remain.

This review also surfaced a further risk worth checking directly rather than assuming away: Appendix A.6’s original 230-row categorization was itself produced before the platform-support defects this review found (the stale 4TU/DANS assumptions, an incomplete dedicated-platform list, a missing DSpace-legacy host count), so any of those original 230 rows that happened to cite one of these same platforms could be mislabeled the same way. Rather than only check the platforms these specific bugs touch, every one of the original 230 rows was rechecked from scratch: each paper’s own text was re-extracted directly from the corpus (paragraph-grouped, not sentence-truncated, avoiding the extraction bug documented earlier in this manuscript), every DOI and URL in that text was identified, and each one’s host was checked directly

against Metacheck’s current supported-platform functions – not against what an earlier comment claimed was or was not supported.

Of the 230, 21 candidate citations resolved to a host now confirmed supported despite the row’s original `UNSUPPORTED_REPO` label; after excluding false positives (a truncated GitHub URL with no repository path, a citation that names a genuinely unrelated, unsupported host despite superficially matching a regex), 9 rows were corrected to `SUPPORTED_PLATFORM_BUG` or `SUPPORTED_PLATFORM_MISSED` at the time of this review. The corrected rows span three recurring patterns: a citation to a genuinely supported host that `repo_check` simply never queried (Dataverse, DataOne/KNB, Zenodo, standard-format Figshare), a supported platform cited via a URL format its matcher does not parse (a Figshare private share-link or collection URL), and a supported platform cited via a non-standard DOI prefix its prefix-matching does not recognise (an alternate Dryad EZID prefix, a non-standard Figshare prefix). None of the corrected rows involve a platform Metacheck genuinely cannot reach at all – every correction is a detection defect on an already-supported platform, consistent with the pattern the original 40-paper review above found. 8 of these 9 papers have since had their underlying defect fixed by later work elsewhere in this project (the same `repo_check-now-finds-it` pattern as above) and no longer appear in the live disagreement set; Table 3 below shows the 1 still remaining in it, with the full original set described narratively here regardless.

Table 6: Table 3. The rows from Appendix A.6’s original 230-paper categorization corrected on full recheck, from `UNSUPPORTED_REPO` to a supported-platform category, still present in the current disagreement set.

Paper	Category	Evidence and verification
10_1111_1365_2435_14709	SUPPORTED_PLATFORM_BUG	Same root cause: the private Figshare share-link format was not recognized by <code>repo_check</code> . RECHECKED: figshare.com is a fully supported platform; the defect is specifically that a private/anonymous Figshare share-link (figshare.com/s/...) uses a different URL format from the standard figshare.com/articles/... pattern <code>repo_check</code> ’s Figshare matcher expects, so the host itself is supported but this specific link format is not recognised -- a supported-platform bug, not an unsupported platform.

Two platforms cited in this set turned out to be genuinely supported by Metacheck despite `repo_check` finding nothing, each traced to a specific, distinct defect rather than a missing backend. **4TU** (`data.4tu.nl`) has dedicated support (`archive-4tu.R`), and the DOI cited (10.4121/uuid:04b25e70-ca3e-489f-9510-36385c1a4104) live-resolves correctly to a real 4TU record. The ID-extractor correctly pulls the UUID from the DOI, but the live 4TU API lookup queries `data.4tu.nl/v2/articles/<uuid>`, which 404s, because 4TU’s own DOI redirect resolves the identical record to a different, numeric article ID that the UUID does not match. This corrects an earlier, incorrect statement elsewhere in this manuscript (Appendix A.6) that this paper’s 4TU fetch failure remained unresolved and should count as `UNSUPPORTED_REPO`; it is a supported-platform bug, not an unsupported platform. **DANS EASY** is one of four separate subject-specific Dataverse installations DANS operates (archaeology/lifesciences/phys-techsciences/ssh), all sharing a single DOI prefix (10.17026).

Metacheck's `.dataverse_host_from_doi()` resolves any DOI under this shared prefix to `archaeology.datastations.nl`, but the paper in this set cites a dataset actually hosted on `lifesciences.datastations.nl` – a genuine Metacheck defect in DOI-to-host resolution for any DOI prefix shared across multiple Dataverse instances, not specific to DANS. A third paper cites the Knowledge Network for Biocomplexity (`knb.ecoinformatics.org`), a confirmed DataOne member node whose DOI live-resolves correctly; the specific reason `repo_check` did not detect it was not further root-caused within the scope of this review, so it is recorded as a supported-platform miss without a fully diagnosed cause, distinct from the two fully diagnosed bugs above.

Table 7: Table 2. Full paper-by-paper review of the 40 previously-unreviewed ‘unsupported repository, not detected’ disagreements: cited platform, live DOI/URL resolution status, and final category.

Paper	Category	Evidence and verification
10_1002_pan3_10149	NO_REPO_DATA_RESTRICTED	Paper text: 'Our raw data are not publicly available for confidentiality reasons, as they contain social, cultural and language data that may not be interpreted outside of their Nyikina context. They will be archived with Griffith University and Walalakoo Aboriginal Corporation.' No repository URL or DOI is given anywhere in the paper -- data are explicitly withheld (Indigenous data sovereignty). There is nothing for <code>repo_check</code> to find; this is not a Metacheck gap of any kind.
10_1111_1365_2745_12930	UNSUPPORTED_REPO	Paper text: 'Data associated with this paper have been deposited in the Environmental Climate Data Sweden depository https://ecds.se/dataset/species-distribution-modelling-data-for-23-dead-wood-inhabiting-bryophytes '. Live-verified the URL resolves (redirects to researchdata.se). researchdata.se is not in any of Metacheck's supported host lists (not Dataverse, DSpace7, DataOne, Figshare, and has no dedicated single-platform module).
10_1002_pan3_10307	UNSUPPORTED_REPO	Paper text: 'Data and code used in the analyses are available at ScienceBase (https://doi.org/10.5066/P9KBZW0G and https://doi.org/10.5066/P9CDA1UN , respectively)'. Live-verified both DOIs resolve to www.sciencebase.gov catalog items. ScienceBase (USGS) is not in any Metacheck supported host list and has no dedicated module.
10_1111_1365_2745_13067	TEXT_EXTRACTION_TRUNCATED	Extracted text is a mid-sentence fragment: '...net online repository (Lopez-Gonzalez, Lewis, Burkitt, & Phillips, 2011; ...)'. No DOI or URL is present in the extracted text at all -- the actual data-availability statement (likely referring to ForestPlots.net , based on the citation) was not captured by GROBID extraction.
10_1111_1365_2435_14226	UNSUPPORTED_REPO	Paper text: 'The data that support the findings of this study are available on the National Tibetan Plateau Data Center https://doi.org/10.11888/Terre.tpd.c.272843 '. Live check: this DOI request failed to resolve (network/redirect issue querying data.tpd.ac.cn from outside China is common). Regardless, tpdc.ac.cn is not in any Metacheck supported host list and has no dedicated module.
10_1002_2688_8319_12315	TEXT_EXTRACTION_TRUNCATED	Paper text: 'The R code, and Google Earth Engine code, used in all analysis and to produce all figures is archived here: https://zenodo.org/record/8347003 '. Zenodo IS a fully supported platform (<code>R/archive-zenodo.R</code>) and the DOI (10.5281/zenodo.8347003, reconstructed from the record ID) live-resolves correctly (HTTP 200, https://zenodo.org/records/10572530). The extracted text inserts a stray space inside the hostname ('zenodo. org'), a GROBID line-wrap artifact, which breaks Metacheck's host-matching regex against the raw citation text. This is a genuine miss of a supported platform caused by text extraction, not a platform-support gap.

Paper	Category	Evidence and verification
10_1111_1365_2745_14315	UNSUPPORTED_REPO	Paper text: 'Novel data collected in this study and compiled from FRED are available through FRED: https://doi.org/10.25581/ornlsfa.014/1459186'. FRED (via ORNL's Sustainable Forestry/SFA data archive) is a niche, domain-specific archive with no entry in any Metacheck supported host list and no dedicated module. Live DOI resolution failed (ornlsfa.014 DOI prefix appears not to resolve through doi.org currently), which is a separate, secondary problem on top of the platform simply being unsupported.
10_1111_2041_210x_14453	UNSUPPORTED_REPO	Paper text: 'FishScale' software ... can be accessed at https://doi.org/10.5066/P13QVR2R. Data available via https://doi.org/10.5066/P9E1DTYB'. Live-verified: the code DOI resolves to code.usgs.gov (a self-hosted GitLab instance on USGS's own domain -- explicitly out of scope per Metacheck's own GitLab module documentation, which matches only gitlab.com); the data DOI resolves to www.sciencebase.gov (unsupported, as elsewhere in this set). Both citations point to unsupported hosts.
10_1111_1365_2745_13529	NCBI_GENBANK	Paper text: 'Data were deposited in the Sequence Read Archive (https://www.ncbi.nlm.nih.gov/bioproject/PRJNA592420)' and 'Sequence data are deposited at NCBI-SRA under the accession number PRJNA592420.' A domain-specific bioinformatics accession (SRA/BioProject), a distinct category from an ordinary data/code repository and outside repo_check's scope by design.
10_1002_pan3_10743	TEXT_EXTRAC- TION_TRUNCATED	Extracted text is entirely a philosophical/conceptual discussion of aesthetics and definitions of 'mess' -- no data-availability statement was captured anywhere in the extracted text for this paper.
10_1111_1365_2664_12991	TEXT_EXTRAC- TION_TRUNCATED	Extraction returned an empty string -- no text was captured for this paper's data-availability section at all.
10_1002_2688_8319_12241	TEXT_EXTRAC- TION_TRUNCATED	Extraction returned an empty string -- no text was captured for this paper's data-availability section at all.
10_1111_1365_2745_13517	TEXT_EXTRAC- TION_TRUNCATED	Extracted text covers only methodological discussion of pollen/charcoal source areas -- no data-availability statement with a URL or DOI was captured anywhere in the extracted text for this paper.
10_1002_pan3_10677	TEXT_EXTRAC- TION_TRUNCATED	Extraction returned an empty string -- no text was captured for this paper's data-availability section at all.
10_1111_1365_2664_13065	UNSUPPORTED_REPO	Paper text: 'Data are available in the United States Department of Agriculture National Agricultural Library (Ag Data Commons) https://doi.org/10.15482/USDA.ADC/1402049'. Live-verified the DOI resolves (HTTP 202) to agdatacommons.nal.usda.gov. Not in any Metacheck supported host list and has no dedicated module.
10_1111_1365_2745_13154	UNSUPPORTED_REPO	Paper text: 'Data available through Digital Commons @ Georgia Southern repository https://doi.org/10.20429/data.2019.01'. Live-verified the DOI resolves (HTTP 200) to digitalcommons.georgiasouthern.edu. A generic bepress Digital Commons instance, not in any Metacheck supported host list.
10_1111_1365_2745_12749	TEXT_EXTRAC- TION_TRUNCATED	Paper text ends mid-sentence: 'Data, including location, growth, and iWUE data are located at ScholarsArchive@OSU' -- the sentence is cut off before any URL or DOI appears, so no identifier was captured for this citation despite a named repository being mentioned.
10_1111_1365_2435_14477	TEXT_EXTRAC- TION_TRUNCATED	Extraction returned an empty string -- no text was captured for this paper's data-availability section at all.

Paper	Category	Evidence and verification
10_1111_1365_2656_13970	UNSUPPORTED_REPO	Paper text: 'Data available from the Apollo-University of Cambridge Repository https://doi.org/10.17863/CAM.97060 '. Live-verified the DOI resolves (HTTP 200) to www.repository.cam.ac.uk . A DSpace-based institutional repository, but not in Metacheck's DSpace7 or DSpace-legacy host lists and has no dedicated module.
10_1111_1365_2664_13892	UNSUPPORTED_REPO	Paper text: 'The entire dataset that supports the findings of this study is available via a permanent open access data repository of the University of Vienna http://phaidra.univie.ac.at/o:1168756 '. Live-verified the URL resolves (HTTP 200). Phaidra (University of Vienna's repository platform) is not in any Metacheck supported host list and has no dedicated module.
10_1111_2041_210x_13110	TEXT_EXTRAC- TION_TRUNCATED	Extracted text covers only general methodological discussion of uncertainty modelling -- no data-availability statement with a URL or DOI was captured anywhere in the extracted text for this paper.
10_1111_1365_2745_14252	TEXT_EXTRAC- TION_TRUNCATED	Extraction returned an empty string -- no text was captured for this paper's data-availability section at all.
10_1111_1365_2664_14700	UNSUPPORTED_REPO	Paper text: 'Data are available from the OFB Digital Repository https://ged.ofb.fr/share/s/XiFQtejFQy28DG6u0TDzoA/folder '. Live-verified the URL returns HTTP 403 (access-restricted share link) on a custom OFB (French biodiversity agency) Nextcloud-style file-share instance. Not in any Metacheck supported host list and has no dedicated module; a bare access-controlled folder link is also not the kind of structured repository API Metacheck's dedicated modules are built to query even where a host is supported.
10_1111_1365_2656_13918	UNSUPPORTED_REPO	Paper text cites four EDI/PASTA data package DOIs (10.6073/PASTA/... and 10.6073/pasta/..., resolving to portal.edirepository.org) for the paper's actual data, plus a properly-formatted Zenodo DOI (10.5281/zenodo.7737746, live-verified HTTP 200) for the analysis code only. EDI/PASTA is not in any Metacheck supported host list and has no dedicated module. Because the data_availability comparison column concerns the data (not the code), and the data are exclusively on EDI/PASTA, this paper is classified as citing an unsupported repository for its data; the Zenodo code citation is correctly formatted and would be expected to be found by Metacheck's code_check separately.
10_1111_2041_210x_13463	UNSUPPORTED_REPO	Paper text: 'The data associated with this manuscript are published on ESS-DIVE, the Department of Energy's data portal...'. No specific DOI is given in the extracted text (the paper states DOIs are 'documented in Supporting Information file 4', not in the main text). ESS-DIVE is not in any Metacheck supported host list and has no dedicated module; Google Earth Engine (also mentioned) is not a structured data repository Metacheck can query as such.
10_1111_1365_2435_13550	NCBI_GENBANK	Paper text: 'The sequences were deposited in NCBI SRA database (accession no. ... SRP218212 and ... SRP068522)'. A domain-specific bioinformatics accession (SRA), outside repo_check's scope by design, consistent with other SRA/GenBank/BioProject cases in this set.
10_1111_1365_2435_12874	TEXT_EXTRAC- TION_TRUNCATED	Extraction returned an empty string -- no text was captured for this paper's data-availability section at all.
10_1111_1365_2664_14787	UNSUPPORTED_REPO	Paper text: 'Data available from the CSIC repository: http://hdl.handle.net/10261/367361 '. Live-verified the URL resolves (HTTP 200) to digital.csic.es . A DSpace-based institutional repository, but not in Metacheck's DSpace7 or DSpace-legacy host lists and has no dedicated module.

Paper	Category	Evidence and verification
10_1111_1365_2656_14227	UNSUPPORTED_REPO	Paper text cites two repositories: 'Guillemot demographic data ... freely available from Heriot Watt research portal https://doi.org/10.17861/8fd6a194-d1a7-469c-8604-e45b0414f8f0 ' (live-verified HTTP 200, researchportal.hw.ac.uk) and 'Puffin and razorbill demographic data ... available upon request ... from the University of Gloucester online repository https://doi.org/10.46289/8FP68KK9 ' (live-verified HTTP 404, likely a stale/incorrect DOI). Neither host is in any Metacheck supported host list, and neither has a dedicated module.
10_1111_1365_2745_13324	UNSUPPORTED_REPO	Paper text: 'Data used in this publication are available in the appendices and archived at the USU Digital Commons: https://doi.org/10.26078/99w7-eb58 '. Live-verified the DOI resolves (HTTP 200) to digitalcommons.usu.edu. A generic bepress Digital Commons instance, not in any Metacheck supported host list.
10_1111_1365_2656_13272	UNSUPPORTED_REPO	Reference-list citation: 'ScholarWorks Data Repository, https://doi.org/10.18122/bio_data/3/boisestate '. Live-verified the DOI resolves (HTTP 200) to scholarworks.boisestate.edu. A generic bepress ScholarWorks instance, not in any Metacheck supported host list.
10_1111_1365_2435_13791	TEXT_EXTRACTION_TRUNCATED	Extraction returned an empty string -- no text was captured for this paper's data-availability section at all.
10_1111_1365_2745_13849	UNSUPPORTED_REPO	Paper text: 'The data are available as a data package on https://www.forestplots.net/ : https://doi.org/10.5521/forestplots.net/2022_1 '. Live-verified the DOI resolves (HTTP 200) to forestplots.net. Forestplots.net is a domain-specific forest-inventory data archive, not in any Metacheck supported host list and has no dedicated module (a separate, unrelated Dryad DOI also appears in this paper's reference list, but only as a citation for a prior, different dataset -- not this paper's own data).
10_1111_1365_2435_14514	UNSUPPORTED_REPO	Paper text: 'The data that support the findings of this study are available from UNL data archive: https://doi.org/10.32873/unl.dr.20230705 '. Live-verified the DOI resolves (HTTP 202) to sandy.unl.edu. Not in any Metacheck supported host list and has no dedicated module.
10_1111_1365_2435_13936	UNSUPPORTED_REPO	Paper text: 'Data are available from the Illinois Data Bank https://doi.org/10.13012/B2IDB-4547091_V1 '. Live-verified the DOI resolves (HTTP 403, access-gated redirect) to databank.illinois.edu. A Hydra/Fedora-based institutional repository, not in any Metacheck supported host list.
10_1111_1365_2664_13845	UNSUPPORTED_REPO	Paper text: 'Data are available in the US Department of the Interior and U.S. Geological Survey's ScienceBase https://doi.org/10.5066/P94B0WUV '. Live-verified the DOI resolves (HTTP 403) to www.sciencebase.gov. ScienceBase is not in any Metacheck supported host list, consistent with the other ScienceBase cases in this set.
10_1111_1365_2664_14229	SOURCE_CITATION_ERROR	Paper text: 'Data available via Unimi Dataverse http://10.0.51.74/RD_UNIMI/5ZXGIV '. Unimi's real Dataverse host (dataverse.unimi.it) IS a supported Dataverse installation (matches .dataverse_host_regex()), but this paper cites a raw internal/private IP address (10.0.51.74, a non-routable RFC 1918 address) instead of the public hostname -- live-verified this address does not resolve externally at all (request failed outright). The paper's own published citation is unusable as given; this is an error in the source text, not a Metacheck platform-support gap or extraction artefact.

Of these 37 papers, 20 genuinely cite an unsupported platform, 2 cite a domain-specific bioinformatics accession outside `repo_check`'s scope, 0 cite a platform Metacheck genuinely supports but failed to detect for the reasons described above, 13 had no usable citation captured

in the paper’s extracted text at all (either empty extraction or a citation cut off before any identifier appeared), 1 explicitly withheld their data with no repository to find, and 1 paper’s own published citation pointed to an internal, non-public address rather than a reachable repository. These per-paper counts feed directly into the corpus-wide reconciling numbers reported in the main text and into Table 1’s totals above.

References

- Cooper, N., & the BES Data and Code Hackathon Group. (2026a). Data and code for MEE paper v1.0. Zenodo. <https://doi.org/10.5281/zenodo.19737911>
- Cooper, N., & the BES Data and Code Hackathon Group. (2026b). Data- and code-archiving in the British Ecological Society journals: Present status and recommendations for future improvements. Methods in Ecology and Evolution, 17, 1954–1966. <https://doi.org/10.1111/2041-210x.70338>
- DeBruine, L., Mesquida, C., Werner, J., & Lakens, D. (2026). Metacheck: Check research outputs for best practices. <https://github.com/scienceverse/metacheck>
- Riedel, N., Kip, M., & Bobrov, E. (2020). ODDPub – a Text-Mining Algorithm to Detect Data Sharing in Biomedical Publications. Data Science Journal, 19(1). <https://doi.org/10.5334/dsj-2020-042>
- Vilhuber, L., & Cavanagh, J. (2025). Report of the AEA Data Editor. AEA Papers and Proceedings, 115, 944–957. <https://doi.org/10.1257/pandp.115.944>
- Wilkinson, M. D., Dumontier, M., Aalbersberg, Ij. J., Appleton, G., Axton, M., Baak, A., Blomberg, N., Boiten, J.-W., Da Silva Santos, L. B., Bourne, P. E., Bouwman, J., Brookes, A. J., Clark, T., Crosas, M., Dillo, I., Dumon, O., Edmunds, S., Evelo, C. T., Finkers, R., ... Mons, B. (2016). The FAIR Guiding Principles for scientific data management and stewardship. Scientific Data, 3(1), 160018. <https://doi.org/10.1038/sdata.2016.18>